

Trustfile: A File security platform that scans and protects files before access

In partial fulfillment for the requirements in Capstone 1 and Research Subject

Taño, Joshua- BSIT 4
Aportadera, Reymart- BSIT 4
Corcino, Eduardo- BSIT 4
Dejaro, Rencel- BSIT 4
Lemeric, Ma. Robella Beatrice- BSIT 4

Cyber Security Track

Second Semester
2025-26

CHAPTER 1

INTRODUCTION

Background of the Study

Nowadays, people use files for school, work, and personal use. Files like documents, PDFs, and image zip or RAR archives are often shared through email, websites, and USB devices. However, some of these files may have malware like viruses, worms, or ransomware that can hurt computers and steal information.

The contemporary digital landscape has experienced a profound transformation over the past two decades, fundamentally changing how individuals, organizations, and institutions create, share, and consume information. The rapid expansion of digital file formats has become a vital element of modern communication, education, commerce, and governance. Documents in formats such as Portable Document Format (PDF), Microsoft Word, Excel spreadsheets, PowerPoint presentations, and compressed archive files like ZIP and RAR, along with image files in JPEG, PNG, and GIF formats, have become the standard mediums for information exchange across virtually all sectors of society. These files are transmitted through multiple channels, including email systems, cloud storage platforms, social media networks, file transfer protocols, removable storage devices like Universal Serial Bus (USB) flash drives, and direct networking connections.

The convenience and efficiency of digital file sharing have facilitated unprecedented levels of collaboration, productivity, and accessibility to information. However, this very convenience has also expanded the attack surface that malicious actors exploit to distribute harmful software.

The threat landscape associated with digital files has evolved considerably in both complexity and sophistication. Malicious software, commonly referred to as malware, includes a wide array of harmful programs such as computer viruses, network worms, Trojan horses, ransomware, spyware, adware, rootkits, and keyloggers. Each category of malware has unique characteristics, propagation mechanisms, and destructive capabilities. For instance, computer viruses attach themselves to legitimate executable files and require user activation to spread, while worms can replicate and traverse networks autonomously without human intervention. Ransomware is one of the most financially devastating forms of malware, as it encrypts victims' files and demands payment for decryption keys. Spyware operates discreetly to collect sensitive information, including passwords, financial data, and personal communications. The economic impact of malware infections is staggering, with global costs estimated at hundreds of billions of dollars annually. These costs encompass direct financial losses, productivity declines, data recovery expenses, reputational harm, and regulatory penalties.

The methods through which malware infiltrates systems have proliferated significantly. Email continues to be the primary delivery mechanism, with phishing campaigns utilizing advanced social engineering techniques to deceive recipients into opening infected attachments. These malicious attachments often disguise themselves as legitimate documents, invoices, shipping notifications, or official correspondence, taking advantage of users' trust and curiosity. Drive-by downloads occur when users visit compromised websites that exploit vulnerabilities in browsers to install malware without their explicit consent. Additionally, removable media devices, such as USB drives, can propagate malware through autorun capabilities or when users inadvertently execute infected files. Peer-to-peer file sharing networks, software piracy sites, and compromised legitimate download platforms serve as major distribution channels for malware-ridden files. The COVID-19 pandemic accelerated the transition to remote work and digital collaboration, thereby broadening the attack surface as employees accessed corporate resources from less secure home networks and personal devices.

Many users download files without adequately assessing their safety, which poses significant risks of system damage and data loss. Although antivirus software exists, much of it functions automatically, removing threats without clearly explaining how file safety is determined. Consequently, users often lack a comprehensive understanding of why specific files may be dangerous. Moreover, malicious files frequently disguise themselves as legitimate ones, making detection challenging. There is also a widespread lack of awareness regarding the proper methods for scanning files prior to opening them.

The human element is undoubtedly the most substantial vulnerability in cybersecurity frameworks. Research consistently demonstrates that the vast majority of successful cyberattacks capitalize on human behavior rather than merely technical flaws. Users often download and execute files from untrusted sources, disable security warnings for quicker access, and fail to verify the authenticity of file origins. Key psychological factors contribute to this behavior, including optimism bias—the belief that they are unlikely to be targeted—time pressure, a lack of technical knowledge, and an inherent trust in familiar file formats and brand names.

The consequences of opening infected files can be catastrophic for both individuals and organizations. System damage can result in corrupted operating systems, deleted files, disabled security software, or complete system compromise, rendering devices inoperable. Data loss can encompass the destruction, theft, or encryption of vital information, such as personal photographs, financial records, intellectual property, customer databases, and proprietary business information. It is essential to recognize and address these vulnerabilities to safeguard personal and organizational data effectively.

Existing antivirus solutions provide essential protection but have significant limitations when it comes to user education and transparency. Traditional antivirus software primarily relies on signature-based detection, comparing file characteristics against databases of known malware signatures. When a threat is detected, the software typically quarantines or deletes the file automatically, offering users minimal information beyond a generic threat name. This lack of transparency fails to educate users about the specific characteristics that made a file dangerous, the potential consequences of executing it, or preventive measures for future encounters. As a result, users remain dependent on the software without developing their own threat recognition capabilities. Additionally, the "black box" nature of automated remediation prevents users from understanding the rationale behind false positives, which can lead to frustration and the disabling of security software.

The sophistication of modern malware obfuscation techniques has made many malicious files virtually indistinguishable from legitimate ones to untrained observers. File extension spoofing allows attackers to disguise executable files as documents by manipulating the visible filename while the actual file type remains executable. Double extension techniques produce filenames such as "document.pdf.exe," where the operating system hides the true extension. Polyglot files contain valid structures for multiple file formats simultaneously, confusing both users and basic file type verification systems. Steganography embeds malicious payloads within seemingly innocuous image, audio, or video files. Packers and crypters compress and encrypt malware to evade signature detection, only decrypting the payload at runtime. Together, these techniques create an environment where visual inspection and simple file property examination are insufficient for protection.

The lack of knowledge regarding pre-access file scanning represents a critical deficiency in general cybersecurity literacy. Most users are unaware of available tools and techniques for assessing file safety before opening them. Concepts such as file hashing, entropy analysis, sandboxing, and multi-engine scanning remain unfamiliar to non-technical users. Educational institutions rarely include practical file security assessment in their curricula, and workplace cybersecurity training typically focuses on high-level policies rather than hands-on technical skills. This knowledge gap leaves users vulnerable and reliant on automated solutions that may struggle against novel threats.

To address this problem, this study proposes the creation of Trustfile, a web-based system that analyzes files using static scanning methods. These methods include file hashing, entropy checking, script detection, abnormal size verification, and file type verification to identify potential threats. We will also integrate API-based malware intelligence services for external threat validation to improve detection accuracy and strengthen overall security against malicious files.

The proposed Trustfile system offers a comprehensive solution to address the current deficiencies in file security practices. By developing a web-based platform accessible through standard browsers, the system removes installation barriers and ensures compatibility across different platforms. Its architecture incorporates multiple complementary static analysis techniques to provide a defense-in-depth strategy against various types of threats.

File hashing is utilized to generate unique cryptographic fingerprints, allowing for rapid identification of known malicious files through database comparisons. Entropy analysis assesses the randomness of file content; elevated entropy values may indicate the presence of encrypted payloads, packed executables, or compressed malicious components. Additionally, script detection identifies embedded JavaScript, VBScript, PowerShell, and other executable code within files that may otherwise appear non-executable. Abnormal file size analysis flags files that deviate significantly from expected sizes based on their stated format, which could suggest the presence of appended malicious code. File type verification checks the internal structures and magic numbers of files to ensure that their actual types align with their extensions and metadata representations.

The integration of external API-based malware intelligence services enhances local scanning capabilities. These services maintain extensive databases of known malware signatures, behavioral indicators, and reputation scores compiled from global threat intelligence networks. By querying these services during the scanning process, Trustfile can identify emerging threats that may not yet be in local databases. This combination of local heuristic analysis and external intelligence validation creates a hybrid detection model that balances the speed and privacy of local processing with the broader coverage of cloud-based threat intelligence. This multi-layered approach significantly improves detection accuracy while minimizing false negatives that could allow malicious files to go undetected.

What sets Trustfile apart from purely automated security solutions is its educational component. Each scan result includes detailed explanations of the specific indicators that triggered threat classifications, the technical significance of these indicators, and actionable guidance for users. This level of transparency transforms each scanning experience into a learning opportunity, gradually building users' knowledge and awareness of threats. By demystifying the detection process and providing contextual information, Trustfile empowers users to make informed decisions and develop their independent security judgment capabilities.

Statement of the Problem

Digital files can be a problem because it is hard for people to know if a digital file is safe to open. Some programs say they can help with files, but these programs do not always tell people how they find the bad things or help people understand what the risks are with digital files.

The fundamental challenge confronting contemporary computer users centers on the inherent difficulty of assessing file safety prior to execution, unlike physical objects that can be examined through direct sensory perception, digital files present opaque interfaces that conceal their true nature and potential effects. The binary data constituting a file is not human-readable without specialized tools and expertise, creating an information asymmetry between users and potential threats. This opacity is compounded by the sophisticated obfuscation techniques employed by malware developers, who invest substantial effort in making malicious files appear benign. The average user possesses neither the technical knowledge nor the analytical tools necessary to perform meaningful pre-execution safety assessments, rendering them dependent on external validation mechanisms.

The existing ecosystem of security software, while providing essential protective functions, exhibits significant shortcomings in user communication and education. Antivirus programs, endpoint protection platforms, and email security gateways typically prioritize threat neutralization over user enlightenment. When a threat is detected, these systems often present minimal information—perhaps a generic threat category name or a numerical risk score—without explaining the specific characteristics that triggered the detection, the potential consequences of execution, or the reasoning behind the recommended action. This opacity creates several negative outcomes: users cannot verify the accuracy of detections, cannot learn to recognize similar threats independently, may distrust the software when false positives occur, and remain perpetually dependent on automated protection without developing their own security awareness. The lack of explanatory depth represents a missed opportunity for security software to function as educational tools that build long-term user resilience.

The disconnect between automated detection and user understanding is particularly problematic in organizational contexts. Employees who encounter blocked files without understanding why may seek alternative methods to access the content, potentially circumventing security controls. Help desk resources are consumed investigating blocked files that users perceive as legitimate. Security incidents may go unreported because users cannot distinguish between normal software behavior and indicators of compromise. The absence of educational content within security workflows perpetuates a cycle of dependency and vulnerability that undermines the overall security posture of organizations.

1. Users cannot easily identify if a file contains malware before opening it.

The first identified problem concerns the fundamental incapacity of typical users to perform meaningful pre-execution threat assessments on digital files. This limitation arises from multiple converging factors. Technically, malware detection requires specialized knowledge of file formats, executable structures, scripting languages, network protocols, and behavioral analysis techniques. The binary content of executable files, the embedded scripts within document macros, and the steganographic payloads hidden in media files are not interpretable through casual inspection. Even examining file metadata, extensions, and properties provides limited protection against sophisticated threats that deliberately manipulate these visible attributes.

The user interface conventions of modern operating systems exacerbate this problem by defaulting to configurations that hide potentially revealing information. File extensions are frequently hidden by default, preventing users from recognizing dangerous double extensions. Icons are determined by file associations rather than actual content, allowing executables to display document icons. Preview panes and thumbnail generation may execute potentially malicious code to render content. The cumulative effect of these design choices is an environment where visual inspection provides minimal security value.

Cognitive and behavioral factors further impede users' threat identification capabilities. Confirmation bias leads users to trust files from expected sources even when anomalies exist. Time pressure and workload demands discourage careful examination. The sheer volume of files encountered daily makes individual scrutiny impractical. Social engineering techniques exploit trust relationships, urgency, and curiosity to override caution. Without systematic training and accessible tools, users cannot reasonably be expected to identify malware through manual inspection.

2. Many systems rely on basic detection methods and may not detect new or hidden threats.

The second problem addresses the technological limitations of existing detection approaches. Signature-based detection, the foundational method employed by traditional antivirus software, maintains databases of known malware fingerprints and flags files matching these patterns. While effective against established threats, this approach fundamentally cannot detect novel malware variants, zero-day exploits, or polymorphic code that modifies its signature with each infection. The temporal gap between malware emergence and signature database updates creates a window of vulnerability during which systems remain unprotected against new threats. Heuristic analysis improves upon signature detection by examining file characteristics and behaviors for suspicious patterns rather than exact matches. However, basic heuristic implementations often rely on simplistic rules that skilled attackers can circumvent. File packing and encryption obscure the code structures that heuristics examine. Legitimate software behaviors may trigger false positives, reducing trust in the system. The arms race between

detection sophistication and evasion techniques continuously challenges the efficacy of heuristic approaches.

The emergence of advanced persistent threats (APTs), fileless malware, and living-off-the-land techniques has further strained traditional detection methods. APTs employ custom malware specifically designed to evade known signatures. Fileless malware operates entirely in memory without creating persistent files for signature matching. Living-off-the-land attacks misuse legitimate system tools and scripts already present on target systems, rendering signature detection irrelevant. These advanced techniques necessitate detection approaches that examine behavioral patterns, anomaly indicators, and contextual risk factors beyond static file characteristics.

3. An automated quarantine and blocking system will be implemented to intercept malicious or abnormally sized files, preventing execution and protecting organizational data from compromise.

The third problem identifies the absence of systematic automated interception mechanisms within typical user workflows. While individual security products may quarantine detected threats, these functions often operate inconsistently across different software platforms, file types, and usage scenarios. Email attachments may be scanned by gateway filters but not by endpoint software. Files downloaded through web browsers may be checked by browser security features, but not by the system antivirus. USB drives may bypass scanning entirely when autoplay is disabled. This fragmented protection landscape creates gaps that sophisticated threats can exploit.

The concept of automated quarantine extends beyond simple file isolation to encompass a comprehensive containment strategy. Effective quarantine systems must intercept files at multiple entry points—email, web downloads, removable media, network shares—ensuring consistent protection regardless of delivery vector. They must classify threats by severity to apply appropriate containment measures, from simple alerting for low-risk anomalies to immediate blocking and administrator notification for critical threats. They must maintain secure isolation environments that prevent accidental or intentional execution of quarantined files while preserving them for forensic analysis. They must integrate with broader security infrastructure to enable coordinated response across organizational systems.

The specific focus on abnormally sized files addresses a frequently overlooked threat indicator. Malware frequently appends payloads to legitimate files, increasing their size beyond normal parameters for the stated format. Alternatively, extremely small files may indicate stub loaders that download additional malicious components. Size analysis provides a simple yet effective preliminary filter that can identify suspicious files warranting deeper inspection. However, implementing size-based detection requires comprehensive databases of normal size ranges for common file types, accounting for version variations, compression levels, and embedded content.

4. There is a lack of systems that explain how malware is detected.

The fourth problem highlights the educational deficiency in existing security solutions. The detection process within most security software operates as an opaque black box, consuming files and producing binary verdicts without revealing the analytical reasoning behind decisions. Users see outcomes—"Threat detected" or "File clean"—without understanding the specific indicators examined, the significance of identified anomalies, or the confidence level associated with the determination. This opacity prevents users from developing threat recognition capabilities, verifying detection accuracy, or making informed risk assessments when encountering edge cases.

The educational value of transparent detection extends beyond individual user benefit to organizational security culture. When employees understand what makes files dangerous, they become more cautious in their file handling practices, more vigilant in recognizing suspicious characteristics, and more effective in reporting potential threats. Transparent systems build trust by demonstrating the factual basis for security decisions, reducing the perception that security software arbitrarily blocks legitimate content. They enable informed exception handling when business needs conflict with security policies, allowing risk-based decisions rather than blanket prohibitions.

The technical communication challenge is significant—explaining complex malware indicators in an accessible language without oversimplifying to the point of inaccuracy. Effective educational content must bridge the gap between technical specificity and user comprehension, employing appropriate analogies, visualizations, and progressive disclosure to accommodate varying knowledge levels. It must contextualize indicators within real-world threat scenarios to demonstrate practical relevance. It must provide actionable guidance that users can apply to future file encounters.

5. Users have limited knowledge about identifying malicious files.

The fifth problem encompasses the broader knowledge deficit regarding file security among general computer users. Cybersecurity education has historically focused on high-level concepts—strong passwords, phishing awareness, software updates—while neglecting practical file assessment skills. Educational curricula from primary through tertiary levels rarely include hands-on training in file analysis, threat indicator recognition, or security tool utilization. Workplace security awareness programs typically emphasize policy compliance over technical competency. The result is a population of computer users who interact with potentially dangerous files daily without possessing the knowledge to evaluate their safety.

This knowledge gap has tangible security consequences. Users cannot distinguish between legitimate and spoofed file extensions. They do not understand the risks associated with macros, scripts, and embedded objects. They are unaware of tools and techniques for independent file verification. They cannot interpret security warnings in context to make

informed decisions. They may disable security features that interfere with workflow without understanding the protection sacrificed. The cumulative effect is a user base that remains perpetually vulnerable to file-based attacks regardless of the sophistication of automated defenses.

Addressing this knowledge deficit requires systematic educational interventions integrated into security workflows. Rather than treating education as a separate training activity, security tools should embed learning opportunities within operational interactions. Each security event should provide explanatory content that builds user knowledge incrementally. Scan results should highlight specific indicators and their significance. Threat classifications should include educational context about the associated malware categories and attack vectors. Over time, this embedded education cultivates a more security-aware user population capable of serving as an additional defensive layer.

Objectives of the Study

General Objective

The general objective of this study is to design and develop TrustFile, a file security platform that will ensure file safety before user access, detect potential threats, and enhance overall system security for businesses or private offices.

The general objective establishes the overarching purpose of the research initiative, encompassing three interconnected dimensions of contribution. The design dimension involves the architectural planning and specification of system components, interfaces, and interactions to achieve the desired security and usability outcomes. Design activities include defining the technical architecture, selecting appropriate technologies and methodologies, specifying functional and non-functional requirements, and creating detailed plans for implementation. The development dimension encompasses the actual construction of the system according to the established design, including coding, integration, testing, and refinement. The enhancement dimension addresses the improvement of security posture for target environments, measuring success through reduced infection rates, faster threat detection, improved user awareness, and strengthened organizational resilience.

The scope of the general objective extends beyond mere technical implementation to encompass the broader goal of security enhancement. Ensuring file safety before user access represents a preventive security paradigm that intercepts threats prior to potential execution, fundamentally different from reactive approaches that respond to infections after damage occurs. Detecting potential threats encompasses both known malware identification through signature and intelligence matching and unknown threat discovery through heuristic and

anomaly analysis. Enhancing overall system security addresses the cumulative effect of improved file handling practices, reduced successful attacks, and elevated user awareness on the security posture of protected environments.

The target beneficiaries—businesses and private offices—reflect the practical application context for the system. Business environments face elevated file-based threat exposure due to high volumes of external communications, diverse file exchange requirements, valuable data assets, and regulatory compliance obligations. Private offices, including government agencies, non-profit organizations, and professional services firms, share similar concerns with potentially fewer resources for comprehensive security infrastructure. By targeting these environments, the research addresses real-world security needs with tangible organizational impact.

Specific Objectives

Specifically, the study aims to:

1. To develop a web-based system that allows users to upload files for security scanning before access.

The first specific objective addresses the foundational technical requirement of creating an accessible file scanning platform. The web-based architecture ensures universal accessibility across operating systems and devices without requiring software installation, lowering barriers to adoption and enabling rapid deployment. Users can access the scanning service from any internet-connected device using standard web browsers, eliminating compatibility concerns and administrative overhead associated with native application distribution.

The file upload mechanism must accommodate diverse file types and sizes while maintaining security during transit. Implementation considerations include secure upload protocols (HTTPS/TLS), file size limitations appropriate to scanning capabilities, format validation to prevent upload of non-scannable content, and protection against upload-based attacks such as buffer overflows or denial of service. The pre-access timing is critical—scanning must complete and present results before the user accesses file contents, ensuring that no potentially malicious content is exposed before the security assessment.

The user interface design must balance simplicity with comprehensiveness, enabling non-technical users to initiate scans effortlessly while providing advanced options for users requiring detailed control. Drag-and-drop functionality, progress indicators, clear status messaging, and intuitive result presentation contribute to a positive user experience. Integration capabilities with common workflows—email clients, file managers, cloud storage—extend the platform's utility beyond standalone usage.

2. Implement a supervised classification system that will categorize threats into SAFE, SUSPICIOUS, and MALICIOUS levels, enabling clear risk communication for non-technical staff.

The second specific objective introduces a structured risk communication framework through supervised threat classification. Supervised classification leverages labeled training data to develop models that assign new observations to predefined categories based on learned patterns. In the context of Trustfile, this involves training classification models on datasets of known safe, suspicious, and malicious files to enable automated categorization of scanned files. The three-tier classification system—SAFE, SUSPICIOUS, and MALICIOUS—provides intuitive risk communication that non-technical users can readily comprehend and act upon. The SAFE classification indicates files that pass all security checks with no detected anomalies, permitting normal access.

The SUSPICIOUS classification identifies files exhibiting anomalous characteristics that do not definitively indicate malicious intent but warrant heightened caution and potentially restricted access pending further review. The MALICIOUS classification applies to files with confirmed or highly probable malicious content, triggering blocking, quarantine, and administrative alerting. The implementation of supervised classification requires careful consideration of feature selection, model training, validation, and ongoing refinement. Relevant features may include file hash matches, entropy values, script presence, size anomalies, type mismatches, API intelligence results, and composite scores derived from multiple indicators. Model selection involves evaluating algorithm performance across metrics, including accuracy, precision, recall, and F1-score, with particular attention to minimizing false negatives (missed threats) while maintaining acceptable false positive rates. Continuous model updating with new threat intelligence ensures classification relevance as the threat landscape evolves.

The communication aspect emphasizes translating technical detection results into actionable guidance for non-technical users. Each classification must be accompanied by clear explanations of the underlying indicators, the rationale for the assigned level, and recommended user actions. Visual design elements—color coding, iconography, severity indicators—reinforce comprehension at a glance. The classification system thus serves dual purposes: automated threat categorization and user education.

3. To apply entropy analysis to detect encrypted, compressed, or potentially malicious files.

The third specific objective focuses on entropy analysis as a specialized detection technique with particular relevance to modern malware characteristics. Information entropy, derived from Shannon's information theory, quantifies the randomness or unpredictability of data within a file. Legitimate files in common formats typically exhibit characteristic entropy profiles reflecting their structural organization, redundancy patterns, and content types. Executable files contain structured headers, predictable instruction patterns, and recognizable code segments. Text

documents feature linguistic patterns and formatting structures. Image files follow established encoding schemes with characteristic compression artifacts.

Encrypted and compressed files exhibit elevated entropy because encryption algorithms deliberately maximize randomness to eliminate patterns that could facilitate cryptanalysis, while compression algorithms replace predictable sequences with shorter representations of random-seeming data. Malware frequently employs packing (compression) and encryption to evade signature detection, resulting in entropy profiles that deviate significantly from normal ranges for the claimed file type. Packers such as UPX, ASPack, and Themida compress executable code to reduce file size and obscure recognizable patterns. Crypters encrypt malicious payloads, decrypting them only at runtime. Both techniques produce files with anomalously high entropy values.

Entropy analysis implementation involves calculating Shannon entropy across file segments or the entire file, comparing results against baseline profiles for the identified file type, and flagging significant deviations for further investigation. However, entropy analysis must account for legitimate sources of elevated entropy, including compressed archives, encrypted containers, multimedia files with inherent high entropy, and certain software protection mechanisms. Contextual analysis incorporating file type, source, and other indicators helps distinguish legitimate high-entropy files from potentially malicious ones.

The detection of encrypted and compressed files serves multiple security purposes beyond malware identification. Encrypted files may indicate attempts to exfiltrate data while evading content inspection. Compressed archives may contain nested malicious files that bypass superficial scanning. Files with unexpected compression may indicate tampering or payload injection. By identifying these characteristics, entropy analysis contributes to comprehensive threat assessment.

4. To connect the system with external API-based malware detection services for detection.

The fourth specific objective addresses the integration of external threat intelligence to enhance detection capabilities beyond locally available resources. API-based malware detection services maintain extensive databases of known malware signatures, behavioral profiles, reputation scores, and threat intelligence accumulated from global sensor networks, security researchers, and automated analysis systems. These services leverage economies of scale and distributed observation that individual organizations cannot replicate independently.

The API integration architecture must balance multiple competing requirements. Query efficiency minimizes latency and resource consumption while maximizing threat coverage. Privacy protection ensures that sensitive file information is not inappropriately exposed to external services—typically achieved through hash-based queries that transmit only file fingerprints rather than file contents. Reliability incorporates fallback mechanisms for service unavailability and graceful degradation when external intelligence is temporarily inaccessible.

Cost management optimizes API usage within budgetary constraints through intelligent caching, batch processing, and tiered querying strategies.

Major API-based services include VirusTotal, which aggregates results from multiple antivirus engines and behavioral analysis tools; MetaDefender, offering multi-scanning and data sanitization; and specialized threat intelligence feeds providing indicators of compromise (IOCs), file reputation data, and contextual threat information. Each service offers distinct strengths in coverage, accuracy, speed, and cost, necessitating careful selection and potential multi-source integration.

The value of external API integration extends beyond direct threat detection to encompass threat landscape awareness, trend identification, and detection capability validation. Discrepancies between local and external detection results highlight areas for local detection improvement. Emerging threat patterns observed across the service's user base provide early warning of new attack campaigns. Historical reputation data reveals files that were initially benign but subsequently identified as malicious, enabling retrospective threat identification.

5. Display clear scan results and an explanation to help users understand the analysis

The fifth specific objective emphasizes the educational and communicative dimension of the Trustfile platform. Clear scan results presentation transforms raw detection data into comprehensible, actionable information that users can understand and apply. This objective recognizes that detection without comprehension fails to build long-term security resilience and may actually increase vulnerability by fostering blind trust in automated systems or, conversely, distrust leading to security circumvention.

Effective result presentation follows established principles of information design and risk communication. Hierarchy organizes information from most critical (threat classification, immediate action required) to supporting details (specific indicators, technical explanations). Visual encoding employs color, size, position, and iconography to convey severity and category at a glance. Plain language replaces technical jargon with accessible terminology while maintaining accuracy. Contextualization relates findings to real-world consequences and user-relevant scenarios. Actionability provides specific, feasible guidance for responding to each classification level.

The explanation component addresses the "why" behind detection results, demystifying the analytical process and building user knowledge. For hash matches, explanations describe the matched malware family, its typical behaviors, and infection vectors. For entropy anomalies, explanations describe what elevated entropy indicates, why it matters, and common legitimate causes. For script detection, explanations identify the specific script type, its capabilities, and why its presence is suspicious in the given file context. For size anomalies, explanations describe expected size ranges and what size deviations may indicate. For type mismatches,

explanations describe how the actual file type differs from the claimed type and common spoofing techniques.

The cumulative effect of clear results and explanations is a user population that gradually develops independent threat recognition capabilities, reducing dependence on automated tools while improving overall security awareness and behavior.

Scope and Delimitation

This study focuses on the development of TrustFile, a web-based System that allows users to scan and analyze files for possible malware. The users can scan individual files within their computer.

The scope statement establishes the boundaries within which the research operates, defining what is included and the context of operation. TrustFile is conceived as a web-based system, meaning it operates through web browsers rather than as a native desktop or mobile application. This architectural choice has significant implications for accessibility, deployment, maintenance, and capability. Web-based systems can be accessed from any device with a compatible browser and internet connection, eliminating platform-specific development and distribution challenges. However, they also face constraints, including browser security sandboxes that limit system-level operations, dependency on network connectivity, and potential performance limitations compared to native applications for resource-intensive operations like deep file analysis.

The scanning functionality encompasses both upload-based scanning, where users submit files through the web interface for server-side analysis, and potentially local scanning capabilities where browser-based or companion components examine files on the user's device before transmission. The scope explicitly addresses individual file scanning rather than batch processing, directory scanning, or continuous monitoring, reflecting the research focus on pre-access assessment of specific files that users intend to open.

The system uses API integration to detect known malware through file hash comparison, while local scanning is used to identify suspicious scripts, patterns, and file type mismatches. Based on the analysis, the system classifies files into threat levels such as Low, Medium, High, and Critical. It also provides scan results and cybersecurity tips to help users understand how malware is detected.

The technical scope encompasses a hybrid detection architecture combining external intelligence and local analysis. API integration for hash comparison leverages the efficiency and comprehensiveness of cloud-based threat intelligence services. File hashing generates

fixed-length cryptographic digests that uniquely identify file contents; comparing these hashes against databases of known malicious files enables rapid identification of previously encountered threats without requiring full file content transmission. This approach is particularly effective against known malware variants and provides immediate results for common threats.

Local scanning complements external intelligence by examining file contents for indicators that may not be present in external databases, particularly for novel or targeted threats. Suspicious script detection identifies embedded executable code within non-executable file formats—JavaScript in PDFs, macros in Office documents, PowerShell in archives—that may indicate malicious intent. Pattern detection applies heuristic rules to identify suspicious code structures, obfuscation techniques, and behavioral indicators associated with malware. File type mismatch detection compares claimed file types (extensions, metadata) against actual internal structures to identify spoofing attempts.

The classification output organizes findings into a four-tier severity hierarchy: Low, indicating minimal risk with normal access permitted; Medium, indicating moderate suspicion requiring additional verification; High, indicating strong malicious indicators with restricted access; and Critical, indicating confirmed malicious content with immediate blocking and alerting. This granular classification enables a differentiated response appropriate to risk levels.

The educational scope extends beyond mere threat identification to include cybersecurity tips that contextualize findings within broader security knowledge. These tips may cover file handling best practices, recognition of common attack vectors, understanding of malware categories and behaviors, and guidance on maintaining system security. By embedding education within operational workflows, the system contributes to long-term security awareness improvement.

However, the system is limited to static file analysis and does not include real-time protection or full system-level security features like commercial antivirus software. It only scans files that are accessible to the user and does not guarantee detection of all types of malware. The system is intended for academic purposes and will be tested using sample files.

The delimitation statements explicitly define what the system does not encompass, managing expectations and clarifying boundaries. Static file analysis examines file contents without executing them, contrasting with dynamic analysis that observes runtime behavior in sandboxed environments. This limitation means the system cannot detect malware that only reveals its malicious nature through execution, such as logic bombs triggered by specific conditions, fileless malware that operates in memory, or malware that downloads additional components post-execution. Static analysis provides safety and efficiency advantages but sacrifices the behavioral insight available through dynamic execution.

The exclusion of real-time protection distinguishes TrustFile from traditional antivirus software that continuously monitors system activities, scanning files as they are accessed, created, or modified. Real-time protection requires deep system integration, kernel-level hooks, and

constant resource consumption that exceed the scope of a web-based scanning platform. Users must explicitly initiate scans rather than receive automatic protection for all file operations. The limitation to user-accessible files acknowledges that the system cannot scan files protected by operating system permissions, encrypted volumes without decryption keys, or files on systems where the scanning service lacks access.

This constraint reflects both technical limitations and privacy considerations—scanning inaccessible files would require elevated privileges that raise security and ethical concerns. The non-guarantee of comprehensive detection acknowledges the inherent limitations of any single security tool. No detection system achieves 100% accuracy against all threat types, and the combination of static-only analysis, limited file access, and specific detection techniques creates acknowledged gaps. This limitation is standard in security research and does not diminish the system's value within its defined scope.

The academic purpose designation clarifies that the system is developed as a research and educational project rather than a commercial product. This affects evaluation criteria, deployment expectations, and liability considerations. Testing with sample files rather than live malware in production environments reflects responsible research practices that prevent accidental harm while still validating detection capabilities.

Significance of the Study

The proposed system will deliver significant benefits to the following stakeholders:

- **End Users** - will experience safer file interactions with automatic pre-access scanning, ensuring that only verified, threat-free files reach their workstations without requiring technical expertise or manual intervention.

The end user stakeholder group encompasses all individuals who interact with digital files in their personal or professional capacities. This includes students exchanging assignments and resources, employees processing work documents, consumers downloading software and media, and general computer users engaging with digital content. The significance for this group centers on the transformation of file security from a specialized technical concern requiring expertise to an accessible, automated service available to all users regardless of technical background.

Safer file interactions result from the systematic application of multiple detection techniques before files reach user workstations. Rather than relying on users to recognize threats or on post-infection remediation, Trustfile intercepts potentially dangerous files at the point of access, preventing exposure before it occurs. This

preventive paradigm fundamentally changes the security model from reactive to proactive, reducing the incidence of successful infections and their associated consequences.

The "without requiring technical expertise" aspect addresses a critical accessibility gap in current security practices. Existing security tools often assume user knowledge that the general population does not possess—understanding of file extensions, recognition of phishing indicators, interpretation of security warnings, and execution of manual scanning procedures. By automating these functions and presenting results in accessible formats, Trustfile democratizes file security, extending protection to users who would otherwise remain vulnerable due to knowledge limitations.

The elimination of manual intervention reduces both the cognitive burden on users and the opportunity for human error. Users are not required to remember to scan files, interpret complex security indicators, or make risk judgments under uncertainty. The system handles these functions consistently and comprehensively, removing the user as a potential point of security failure while still informing and educating through result explanations.

- **The Organization** - will demonstrate a responsible and intelligent approach to file security, establishing TrustFile as a practical model for protecting digital assets in government office environments.

The organizational stakeholder group includes businesses, government agencies, educational institutions, non-profit organizations, and any entity that manages digital assets and faces file-based security risks. The significance for organizations extends beyond direct security improvement to encompass reputational, operational, and strategic dimensions.

Demonstrating responsible file security involves implementing visible, effective controls that protect organizational data, maintain operational continuity, and fulfill duty-of-care obligations to stakeholders. In an era of increasing regulatory scrutiny regarding data protection—GDPR, HIPAA, PCI-DSS, and sector-specific regulations—demonstrable security measures provide evidence of compliance efforts and due diligence. Trustfile contributes to this demonstration by providing auditable scanning records, consistent security application, and documented threat detection capabilities.

The "intelligent approach" characterization emphasizes the system's integration of multiple detection techniques, external intelligence, and educational components rather than reliance on single-point solutions. This multi-layered strategy reflects security best practices and positions the organization as adopting sophisticated, contemporary protection methodologies. The intelligence aspect also encompasses the system's

learning and adaptation capabilities, improving detection accuracy over time through accumulated experience and updated threat intelligence.

The reference to government office environments specifically acknowledges the particular relevance of file security in public sector contexts. Government agencies handle sensitive citizen data, classified information, and critical infrastructure controls that make them high-value targets for cyberattacks. Budget constraints, legacy systems, and workforce diversity often complicate security implementation in these environments. Trustfile's web-based, user-friendly, and cost-effective approach addresses these constraints, providing a practical model that can be replicated and adapted across government contexts.

Establishing Trustfile as a practical model involves not only demonstrating technical viability but also documenting implementation processes, evaluating effectiveness, and disseminating findings to enable broader adoption. The research contributes to the body of knowledge regarding accessible file security solutions, providing a template that other organizations can adapt to their specific contexts.

- **Future Researchers** - They will use this study as a reference. They will build on it to advance learning systems. This will also help improve security applications in research and development efforts.

The future researcher stakeholder group encompasses academic researchers, graduate students, industry researchers, and developers who will conduct subsequent investigations in file security, malware detection, web-based security tools, and related domains. The significance for this group centers on the research's contribution to cumulative knowledge and its role as a foundation for further advancement.

As a reference, this study provides documented methodology, implementation details, evaluation results, and lessons learned that subsequent researchers can consult, critique, and build upon. The documented approach to hybrid detection architecture, API integration strategies, user interface design for security tools, and educational content integration offers specific technical guidance. The evaluation framework provides benchmarks against which future systems can be compared.

Building on this study to advance learning systems specifically references the potential for extending Trustfile's capabilities through machine learning and artificial intelligence techniques. The supervised classification system described in the objectives provides a foundation that can be enhanced with more sophisticated algorithms, larger training datasets, and advanced feature engineering. The static analysis techniques can be supplemented with dynamic analysis, behavioral modeling, and automated reverse engineering. The web-based architecture can be extended with mobile applications, browser extensions, and enterprise integration capabilities.

Improving security applications in research and development reflects the broader impact of academic research on practical security tools. Findings regarding effective detection techniques, user interface designs, educational approaches, and system architectures inform commercial product development, open-source projects, and organizational security implementations. The research contributes to the innovation pipeline that translates academic insights into practical protections.

Definition of Terms

- **Abnormal File Size**

Refers to a file that exceeds the expected size for its content or context, potentially indicating embedded malware or any malicious files.

A file that is too big for what it's supposed to be can be a problem. This is because it might have things like malware inside it. People who work with computer security and digital forensics think that a file size that is not normal is a warning sign. They use this sign to find files that might be bad when they are looking at them.

Every type of file has its rules about how big it should be. This depends on what's, inside the file how complicated it is and how it is compressed. For example a simple text document is usually a few kilobytes or a few megabytes.. Big pictures and videos can be really big because of how clear they are and how long they are.

If a file is much bigger or smaller than it should be that is a sign. It might mean that someone has added code to the file. This can make the file do things to your computer. On the hand if a file is too small it might be a special kind of file that downloads bad things from the internet when you open it.

- **API (Application Programming Interface)**

is a set of rules or protocols that enables software applications to communicate with each other to exchange data, features, and functionality.

APIs are like a set of rules that let software applications talk to each share data and features. In software engineering and cybersecurity APIs are the building blocks that let different systems and platforms work together seamlessly. An API is like a contract that says how software components should interact with each other. It says what operations are available what input is needed what the response will be and how to handle errors.

- **Threat Classification**

The automated categorization system will assign risk levels (LOW, MEDIUM, HIGH, CRITICAL) based on file behavior and characteristics.

The system that automatically categorizes things will decide how risk a file has. It can say if a file is MEDIUM, HIGH or CRITICAL risk. This helps the security team know what to do how to tell people about the risk. The system uses computers to look at lots of things about the file like if it matches something in a database if it has code that might be hiding something or if it looks like it is pretending to be something else.

The system has four levels of risk: LOW, MEDIUM, HIGH and CRITICAL. The LOW level is for files that seem safe. The MEDIUM level is for files that seem a little suspicious. We are not sure if they are bad. The HIGH level is for files that we think are probably bad like if they have code that we know is malicious. The CRITICAL level is for files that we know are very bad. Could hurt us right away.

- **Critical Threat Level**

The highest risk classification that will be assigned to files exhibiting confirmed malicious behavior, requiring immediate blocking and administrative alert.

The highest risk classification is for files that're definitely bad and need to be stopped right away. This is the level of threat that Trustfiles system can assign to a file. It means the file is very dangerous and can hurt the organizations security, data and ability to work. To get this classification the system needs to be very sure the file is bad by finding several signs that it is malicious or by matching it to known bad files.

- **High Threat Level**

A classification that will indicate strong suspicious characteristics in a file, warranting restricted access and detailed administrative review.

When a file has suspicious characteristics it gets a special classification. This classification means the file needs to be looked at closely by administrators and that people should not be able to access it easily. The high threat level is a classification that Trustfile uses to figure out how risky a file is. It is for files that're not definitely bad but that still seem suspicious.

This classification is useful because some bad files are made to avoid being detected. They do not have the signs of being bad but they still do things that seem suspicious. For example a file might have a lot of data or it might have code that can do bad things. It might not be the type of file or it might have come from a bad source. If a file has a lot of these signs it gets a high classification.

- **Medium Threat Level**

A classification that will indicate moderate suspicious activity, prompting additional verification steps before file release.

This classification is for things that seem a little suspicious so we need to do some checks before we let people use the file. The medium threat level is for files that're not quite right but we are not really sure if they are bad or not. This is because looking at files for security problems is not always easy and sometimes we have to make a judgment call.

We might see some things that make us think a file is suspicious like it has some numbers that are higher than usual for that kind of file or the information about the file does not match up. Maybe the file can do some things that it should not be able to do. It was made recently and is being sent around in a strange way.

This classification is also for files that come from places that we're not really sure about or files that act like other files that we know are bad but we are not really sure. The medium classification means we have to do some checks to be sure everything is okay.

- **Low Threat Level**

A classification that will indicate minimal risk, allowing normal file access while maintaining routine monitoring.

A classification that shows something is not risky means we can use the file like we normally do but we still keep an eye on it. The low threat level is the classification that Trustfile uses when a file seems completely safe. This happens when the file passes all the security checks and does not have anything about it. It has the structure, metadata and content and it behaves like it should. We need to remember that just because something is labeled as not risky it does not mean it is completely safe. When we check something for security we are looking at probabilities, not guarantees.. Because new threats are always coming up sometimes something bad might slip through until we find out about it and update our security.

- **Malware**

Refers to any intrusive software developed by cybercriminals (often called hackers) to steal data and damage or destroy computers and computer systems.

This malware can cause a lot of damage. It can steal information, account numbers and secret company information. It can also break computers by deleting files messing up data and disrupting services. In some cases it can even destroy computers by corrupting the firmware or hurting the parts.

Malware can also let hackers get into computer systems and stay there move around the network and even use the compromised computers to attack computers. To understand malware we categorize it in ways. We look at how it spreads what it does and how it stays hidden. For example some malware attaches to files some can spread on its own. Some looks like regular software but is actually bad.

- **Pre-Access Scanning**

The process of scanning a file before it is opened to check for possible threats.

Scanning a file before it is opened is an important step to check for possible threats. This is called -access scanning. It is a security measure that checks files for threats before a user can open them. This check is very important because if a file has code it can cause a lot of damage as soon as it is opened.

Pre-access scanning checks the file for threats before it is opened. This way it can prevent code from running and causing damage. The scanning process uses different techniques to check the file. It checks the files structure, content and metadata without running the file. It also uses algorithms and machine learning models to detect malicious code.

- **Worm**

A computer worm is a type of malware that can copy itself and spread to computers and networks without needing a file on the computer or any help from the user. Computer worms are very bad. Can cause a lot of damage because they can spread on their own across networks, systems and different parts of the world very quickly without needing to be attached to a file or for the user to do anything.

This is different from viruses which need someone to open a file to start spreading. Computer worms use weaknesses in the network, bad software, weak security settings and problems with how the network talks to systems to get into new computers copy their bad code and start new instances that keep spreading on their own. This ability to spread on their own has made some of the cyberattacks in history happen, like the Morris Worm in 1988 that got into about 6,000 computers in just one day.

- **Virus**

Computer viruses are software that attach themselves to good files and programs to spread from one computer to another. They are like viruses that need a host to multiply and spread. Computer viruses have been around for a time and are the original type of bad software. They work by attaching

themselves to files and programs which then get changed to include the bad code. This bad code can run when someone opens the file without knowing it.

The way computer viruses spread is pretty complicated. First they look for files to attach themselves to. They do this by scanning the computers files watching what the computer is doing or intercepting system calls. They usually target files that can be run, like programs or documents. Then they insert their code into the good file. They can do this in a few ways like adding the bad code to the end of the file or mixing it in with the good code. The virus has to be careful not to mess up the file too much or it will get caught. It also has to make sure its bad code runs at the time.

Computer viruses can spread when people share files. This can happen through email, removable media like USB drives or files downloaded from websites. When someone opens a file on a new computer the bad code runs and infects other files on that computer. This can make the virus spread to computers through network shares or email. Because people have to do something to spread the virus like open a file it can be harder to catch than other types of bad software.

- **Entropy**

A mathematical concept borrowed from information theory that measures the randomness or unpredictability of data within a file. In the context of malware detection, entropy analysis is used to identify files that have been encrypted, compressed, or packed—techniques commonly employed by malware authors to hide malicious code from traditional antivirus scanners. When a file's entropy value is unusually high, it suggests that the data inside is highly randomized, which is characteristic of encrypted payloads, packed executables, or compressed malicious components. Legitimate files such as text documents, images, or standard executables typically exhibit lower entropy because they contain structured, predictable patterns.

However, malware developers use tools called "packers" and "crypters" to compress and encrypt their malicious code, which dramatically increases the file's entropy. By calculating the Shannon entropy of a file, security systems like Trustfile can flag suspicious files that deviate from normal entropy ranges for their stated file type, providing an early warning that the file may contain hidden threats. It is important to note that some legitimate files—such as compressed archives, multimedia files, or certain software protection mechanisms—can also exhibit high entropy, so entropy analysis must be combined with other detection methods to avoid false positives.

- **Heuristic Analysis**

Scanning technique used by antivirus and security software to detect potentially new and unknown malware variants by examining their behavior and characteristics rather than relying on exact signature matches. Unlike signature-based detection, which can only identify malware that has been previously discovered and cataloged, heuristic analysis applies rules and algorithms to identify suspicious patterns, code structures, and behavioral indicators that are commonly associated with malicious software.

For example, a heuristic scanner might flag a program that attempts to modify system files, disable security software, or establish hidden network connections—behaviors that are typical of malware but rare in legitimate applications. The advantage of heuristic analysis is its ability to catch "zero-day" threats and polymorphic malware that have not yet been added to signature databases.

However, heuristic analysis is not without challenges: it can sometimes misidentify legitimate software as malicious (producing "false positives"), especially if the legitimate program uses unusual coding techniques or performs actions that resemble malware behavior. Skilled attackers can also design malware specifically to evade heuristic rules by mimicking legitimate software behaviors or using advanced obfuscation techniques. In the Trustfile system, heuristic analysis complements signature-based detection and API-based intelligence to create a multi-layered defense that improves overall detection accuracy.

- **Sandboxing**

A security mechanism that isolates potentially malicious programs or files in a controlled, restricted environment—called a "sandbox"—where they can be executed and observed without risking damage to the host system or network. The sandbox acts as a virtual container that simulates a real computer environment, complete with an operating system, file system, and network connections, but completely separated from the actual production environment.

When a suspicious file is placed in a sandbox, security analysts can monitor its behavior in real-time: what files it tries to access, what registry changes it attempts to make, what network connections it establishes, and what processes it spawns. This dynamic analysis reveals the true nature of the file, including any malicious actions that would not be visible through static analysis alone.

Sandboxing is particularly effective against sophisticated threats such as fileless malware, logic bombs, and polymorphic viruses that only reveal their malicious payload during execution. However, sandboxing has limitations: some advanced malware can detect when it is running in a virtual environment and will deliberately alter its behavior or refuse to execute to avoid detection. Additionally, sandboxing requires significant computational resources and time, making it impractical for real-time scanning of every file. In the Trustfile system, sandbox execution is mentioned as a potential verification step for files classified as suspicious, providing a deeper layer of analysis before allowing user access.

- **Polyglot Files**

Sophisticated type of malicious file that contains valid structures for multiple file formats simultaneously, allowing them to function as different types of files depending on how they are opened or interpreted. For example, a polyglot file might be opened as a legitimate PDF document in a PDF reader, displaying normal content to the user, while simultaneously being executable as a malicious program when processed by an operating system or specific application.

This dual-nature makes polyglot files particularly dangerous because they can bypass simple file type verification checks that only examine file extensions or basic metadata. Attackers create polyglot files by carefully crafting the binary data so that it satisfies the structural requirements of two or more file formats—for instance, embedding executable code within the header or trailer sections of an image file, or exploiting overlapping format specifications. When a user opens what appears to be a harmless image or document, the underlying executable code may trigger, installing malware without the user's knowledge.

Polyglot files challenge traditional security tools because they pass basic file type checks while harboring hidden malicious functionality. The Trustfile system addresses this threat through file type verification that examines internal file structures and "magic numbers" (unique byte sequences that identify file types) to confirm that the actual file type matches its claimed format, helping to detect and block polyglot-based attacks.

- **Steganography**

The practice of concealing secret information—including malicious code or data—within seemingly innocuous files such as images, audio files, video files, or text documents, in a way that avoids detection by casual observers or automated security scanners. Unlike encryption, which scrambles data to make it unreadable without a key, steganography hides the very existence of the hidden data by embedding it within the normal content of a carrier file.

In the context of malware, attackers use steganography to embed malicious payloads within image files (JPEG, PNG, GIF), audio files (MP3, WAV), or video files, making the file appear completely normal when viewed or played. The hidden payload can be extracted and executed by a separate malicious program, or the steganographic file itself can be designed to trigger malicious behavior when processed by a vulnerable application. Common steganographic techniques include least significant bit (LSB) insertion, where the hidden data is encoded in the least important bits of each pixel's color value, or metadata manipulation, where data is hidden in file headers or comment fields.

Steganography poses a significant challenge for malware detection because the carrier file itself is not inherently malicious—it is a legitimate image, audio, or video file—and standard signature-based scanners may not detect the hidden payload unless they specifically look for steganographic patterns. The Trustfile system acknowledges steganography as a modern obfuscation technique that makes files "virtually indistinguishable from legitimate counterparts to untrained observers," highlighting the need for advanced detection methods beyond simple visual inspection.

- **Packers and Crypters**

Tools used by malware developers to compress, encrypt, or obfuscate malicious code, making it harder for antivirus software to detect and analyze. A packer (also called a compressor or executable packer) compresses the malicious executable file, reducing its size and obscuring its internal code structure. When the packed file is executed, a small decompression routine embedded in the file unpacks the original malicious code into memory and runs it. Common packers include UPX, ASPack, and Themida. Packers are sometimes used legitimately to reduce file sizes or protect software against reverse engineering, but they are frequently abused by malware authors to evade signature-based detection. A crypter, on the other hand, encrypts the malicious payload so that the actual harmful code is completely hidden until runtime.

The crypter embeds a decryption stub in the file that decrypts the payload in memory when the file is executed, leaving no trace of the original malicious code on disk. Crypters can be "scantime" (decrypting when the file is scanned) or "runtime" (decrypting only when executed), with runtime crypters being more effective at evading detection. Both packers and crypters significantly alter the file's appearance, making signature-based detection ineffective because the packed/encrypted file no longer matches known malware signatures. They also increase the file's entropy, which is why entropy analysis is a valuable complementary detection technique.

The document notes that "packers and crypters compress and encrypt malware to evade signature detection, only decrypting the payload at runtime," underscoring their role in the ongoing arms race between malware developers and security researchers.

- **Rootkits**

A particularly insidious category of malware designed to gain deep, persistent access to a computer system while actively hiding their presence from users and security software. The term "rootkit" derives from "root" (the highest-level administrator account in Unix/Linux systems) and "kit" (the collection of tools used to maintain access). Rootkits operate at various levels of the system architecture, from user-mode rootkits that manipulate operating system APIs and system libraries, to kernel-mode rootkits that inject malicious code directly into the operating system kernel—the core component that controls all system operations. Kernel-mode rootkits are especially dangerous because they operate with the highest level of system privileges, allowing them to intercept and modify system calls, hide files and processes, disable security software, and manipulate network traffic.

Some advanced rootkits even infect the system's firmware or boot sector, loading before the operating system itself and making them virtually impossible to detect or remove using standard antivirus tools. Rootkits are often used in combination with other malware types—such as keyloggers, spyware, or backdoors—to maintain long-term, undetected access to compromised systems. They enable attackers to steal data, monitor user activities, install additional malware, or use the compromised system as part of a botnet. Because rootkits actively conceal their presence by subverting the very mechanisms used to detect them, specialized tools such as behavioral analysis, memory forensics, and hardware-based integrity checks are often required for detection. The document mentions rootkits as part of the broad spectrum of malware that threatens digital file security.

- **Keyloggers**

Type of spyware malware designed to covertly record every keystroke typed by a user on their keyboard, capturing sensitive information such as passwords, credit card numbers, personal messages, and other confidential data without the user's knowledge or consent. Keyloggers can operate at different levels: hardware keyloggers are physical devices inserted between the keyboard and the computer, while software keyloggers are malicious programs installed on the victim's system.

Software keyloggers can be implemented as kernel-level drivers that intercept keyboard input at the operating system level, or as user-level applications that monitor window events and clipboard contents. Some sophisticated keyloggers also capture screenshots, record audio from microphones, or log mouse clicks in addition to keystrokes. The captured data is typically stored locally on the compromised system and periodically transmitted to the attacker via email, FTP, or command-and-control servers. Keyloggers are particularly dangerous because they bypass many security measures—users can have strong passwords, encrypted connections, and updated antivirus software, but if a keylogger is recording their keystrokes, all of that security is compromised.

They are often distributed through phishing emails, malicious downloads, or bundled with other malware. Detection of keyloggers can be challenging because they often masquerade as legitimate system processes or drivers. The document includes keyloggers in its list of malware types that threaten file and data security, emphasizing the need for comprehensive scanning and detection systems like Trustfile.

- **Fileless Malware**

Type of malicious software that operates entirely in a computer's memory (RAM) without writing any files to the hard disk, making it extremely difficult to detect using traditional file-based security tools. Unlike conventional malware that installs executable files, registry entries, or other persistent components on the system, fileless malware leverages legitimate system tools and built-in operating system features—such as PowerShell, Windows Management Instrumentation (WMI), or macros in Office documents—to execute malicious commands directly in memory. Because it leaves no persistent files on disk, fileless malware bypasses signature-based detection, file integrity monitoring, and standard antivirus scans that focus on examining stored files.

It also evades forensic analysis that relies on examining hard drive contents after an incident. Fileless malware often exploits vulnerabilities in legitimate software or uses social engineering to trick users into enabling macros or scripts that launch the attack. Once active in memory, it can perform all the malicious actions of traditional malware: stealing credentials, moving laterally across networks, establishing backdoors, and exfiltrating data. The document explicitly mentions fileless malware as a challenge for traditional detection methods, noting that "fileless malware operates entirely in memory without creating persistent files for signature matching," which renders signature-based detection irrelevant.

Detecting fileless malware requires behavioral monitoring, memory analysis, and endpoint detection and response (EDR) tools that can observe suspicious activities in real-time rather than relying on static file scans.

- **Advanced Persistent Threats (APTs)**

Long-term cyberattack campaigns typically conducted by well-funded and highly skilled threat actors—often nation-state groups, organized cybercriminal organizations, or corporate espionage teams—targeting specific organizations or entities with the goal of stealing sensitive data, disrupting operations, or maintaining persistent access for extended periods. The term "advanced" refers to the attackers' use of sophisticated techniques, custom-developed malware, and zero-day exploits; "persistent" indicates their determination to maintain long-term access to the target environment; and "threat" underscores the serious risk they pose. APTs differ from opportunistic attacks (such as mass-distributed ransomware or common viruses) in that they are highly targeted, carefully planned, and executed in multiple stages over months or even years.

A typical APT attack follows a "kill chain" pattern: initial reconnaissance, initial compromise (often through spear-phishing or watering-hole attacks), establishment of foothold, privilege escalation, lateral movement across the network, data exfiltration, and long-term persistence. APTs frequently employ custom malware specifically designed to evade the target organization's existing security controls, making them particularly challenging to detect with standard signature-based antivirus. The document mentions APTs as a category of threat that has "further strained traditional detection methods," necessitating detection approaches that examine behavioral patterns, anomaly indicators, and contextual risk factors beyond static file characteristics.

- **Living-off-the-Land**

Is a stealthy attack technique where threat actors use legitimate system tools, utilities, and software already present on the target computer—rather than bringing in custom malware—to carry out malicious activities. By leveraging trusted, pre-installed tools such as PowerShell, Windows Management Instrumentation (WMI), Command Prompt, Task Scheduler, or built-in scripting languages, attackers can blend their malicious activities with normal system operations, making detection extremely difficult. For example, an attacker might use PowerShell—a legitimate and powerful administrative tool built into Windows—to download additional malicious components, execute commands, or establish persistence, all while appearing as normal administrative activity to security software.

Similarly, attackers might use Windows' built-in remote desktop protocol (RDP) for lateral movement, or legitimate file transfer tools for data exfiltration. Because these tools are inherently trusted and frequently used by system administrators, security tools often whitelist them or allow their execution without scrutiny. The "living-off-the-land" approach renders traditional signature-based detection largely ineffective because no malicious files are introduced to the system—only

legitimate tools are abused for malicious purposes. The document explicitly identifies living-off-the-land attacks as a technique that "misuse legitimate system tools and scripts already present on target systems, rendering signature detection irrelevant," highlighting the need for behavioral analysis and anomaly detection in modern security architectures like Trustfile.

- **Indicators of Compromise (IOCs)**

Are forensic artifacts, observable evidence, or pieces of data that suggest a computer or network has been breached or is currently under attack by malicious actors. IOCs serve as the "fingerprints" of cyberattacks, helping security analysts and automated systems identify, investigate, and respond to security incidents.

Common types of IOCs include malicious file hashes (unique cryptographic identifiers of known malware files), suspicious IP addresses or domain names associated with command-and-control servers, unusual network traffic patterns, registry modifications, specific file paths or filenames used by malware, email addresses or subject lines from phishing campaigns, and digital certificates used to sign malicious software. IOCs are collected from various sources: security researchers analyzing malware samples, threat intelligence feeds from security vendors, incident response investigations, honeypots (decoy systems designed to attract attackers), and global security communities. Once identified, IOCs are distributed through threat intelligence platforms, security information and event management (SIEM) systems, and endpoint protection tools to enable proactive detection and blocking of known threats.

The document mentions "specialized threat intelligence feeds providing indicators of compromise (IOCs), file reputation data, and contextual threat information" as part of the external API-based services that Trustfile integrates with, demonstrating how IOCs enhance the system's detection capabilities beyond local analysis.

- **SIEM (Security Information and Event Management)**

Is a comprehensive security technology that combines two primary functions—Security Information Management (SIM) and Security Event Management (SEM)—to provide real-time analysis, correlation, and reporting of security alerts and events generated by network hardware, software applications, and security devices across an organization's IT infrastructure.

The SIM component focuses on collecting, storing, and analyzing log data from various sources over time, enabling historical analysis, compliance reporting, and forensic investigation. The SEM component focuses on real-time monitoring, correlation of events across multiple systems, and automated alerting when suspicious patterns or security incidents are detected.

A SIEM system aggregates data from firewalls, intrusion detection systems, antivirus software, servers, databases, applications, and network devices, normalizing the data into a common format and applying correlation rules to identify relationships between events that might indicate a coordinated attack. For example, a SIEM might correlate a failed login attempt from an unusual IP address, followed by a successful login, followed by abnormal data access patterns—all occurring within a short timeframe—to flag a potential account compromise. SIEMs also provide dashboards, visualizations, and automated reporting that help security teams understand their security posture, investigate incidents, and demonstrate compliance with regulatory requirements. The document mentions SIEM integration as part of the alerting mechanism for critical threat detections in the Trustfile system, indicating that the system can feed its detection results into broader organizational security operations workflows.

- **Convolutional Neural Networks (CNNs)**

Are a specialized class of deep learning artificial neural networks primarily designed for processing and analyzing visual data, such as images, but increasingly applied to other domains including malware detection. CNNs are inspired by the biological visual cortex, where neurons respond to specific regions of the visual field. The architecture of a CNN consists of multiple layers: convolutional layers that apply filters (kernels) to detect features such as edges, textures, and patterns; pooling layers that reduce dimensionality and computational complexity; and fully connected layers that perform final classification. In the context of malware detection, CNNs are used in an innovative approach where binary executable files are converted into grayscale images—by interpreting file bytes as pixel intensity values—and then fed into the CNN for classification.

The CNN automatically learns to recognize visual patterns that distinguish malware from benign software, such as structural differences in code sections, data segments, and padding areas. The document cites research by Caluza (2024) that achieved an impressive 99.86% validation accuracy using this CNN-based image representation approach on the Mallmg dataset, which contains over 6,414 malware variants and 2,050 benign files. This approach is particularly valuable because it can potentially detect packed, encrypted, or obfuscated malware that evades traditional signature-based methods, as the packing and encryption processes leave distinctive visual patterns in the image representation. While Trustfile's initial scope focuses on static analysis techniques, the reviewed literature provides strong justification for future incorporation of CNN-based classification as an advanced detection layer.

- **Autoencoders**

A type of artificial neural network used in unsupervised machine learning, designed to learn efficient representations (encodings) of input data by training the network to copy its input to its output through a compressed intermediate representation. An autoencoder consists of two main components: an encoder that compresses the input data into a lower-dimensional latent space (bottleneck), and a decoder that reconstructs the original input from this compressed representation.

The network is trained to minimize the difference between the original input and the reconstructed output, forcing it to learn the most salient features of the data. In cybersecurity and malware detection, autoencoders are used for several purposes: anomaly detection (where the autoencoder learns patterns of normal files and flags files with high reconstruction error as potentially anomalous), dimensionality reduction (compressing high-dimensional file features into manageable representations), and denoising (learning to reconstruct clean data from corrupted inputs). The document mentions autoencoders as part of the deep learning techniques identified by Aslan and Samet (2020) that show promise for detecting both known and unknown malware variants. Autoencoders are particularly useful for detecting novel threats because they do not rely on labeled training data of known malware—instead, they learn the "normal" characteristics of benign files and identify deviations that may indicate malicious content.

- **Polymorphic and Metamorphic Malware**

Two advanced evasion techniques used by malware authors to circumvent signature-based detection systems. Polymorphic malware encrypts its malicious payload with a variable encryption key and appends a decryption routine that changes with each infection. While the underlying malicious code remains functionally identical, the encrypted body and decryption routine appear completely different in each generation, producing a unique file hash and byte pattern every time.

This means that signature-based antivirus software, which relies on matching files against known malware fingerprints, cannot recognize subsequent variants of the same polymorphic malware family because each instance looks different. Metamorphic malware goes a step further by completely rewriting its own code with each generation, changing not just its appearance but its internal structure, instruction sequences, and control flow while maintaining the same malicious functionality.

Metamorphic engines use code obfuscation techniques such as register renaming, instruction substitution, code permutation, and garbage code insertion to produce functionally equivalent but structurally distinct variants. Both techniques represent the ongoing arms race between malware developers and security researchers: as detection systems improve at identifying encrypted or packed malware, attackers develop more sophisticated code transformation techniques. The document explicitly mentions polymorphic and metamorphic malware as challenges that "have further strained traditional detection methods," necessitating advanced detection approaches such as behavioral analysis, heuristic detection, and machine learning that can identify malware based on what it does rather than what it looks like.

- **Bootkits**

Advanced and particularly dangerous category of malware that infects the master boot record (MBR), volume boot record (VBR), or boot sector of a computer's hard drive, allowing the malware to load before the operating system itself during the system startup process. By infecting the boot process, bootkits gain control of the system at the earliest possible stage, enabling them to bypass or subvert security measures that load later, including antivirus software, host intrusion prevention systems (HIPS), and operating system security features. Once active, bootkits can load rootkits, disable security software, manipulate system files, and establish persistent backdoors that survive operating system reinstallation. Because they operate below the operating system level, bootkits are extremely difficult to detect and remove using standard security tools that run within the operating system environment. Detection typically requires specialized tools that can scan the boot sector from outside the infected operating system—such as bootable rescue disks, hardware-based security modules, or firmware integrity checks. The document includes bootkits in its taxonomy of malware types, alongside rootkits, highlighting the need for comprehensive security approaches that address threats at all levels of the system architecture, not just the application or operating system layers.

- **Logic Bombs**

A type of malicious code secretly inserted into a software system or file that remains dormant and harmless until a specific triggering condition is met, at which point it executes its destructive payload. The trigger conditions can be based on various factors: a specific date or time (time bombs), the absence of a particular file or user account, a certain number of program executions, a specific system configuration, or even external signals from a command-and-control server. Until triggered, logic bombs are virtually undetectable because they do not exhibit any malicious behavior—they appear as normal, legitimate code or files.

This makes them particularly insidious because they can pass through security scans, code reviews, and antivirus checks without raising suspicion. Famous examples include the case of a disgruntled employee who planted a logic bomb in a company's financial system that would delete critical data if his employee ID was ever removed from the payroll system. The document mentions logic bombs as a limitation of static analysis, noting that "the system cannot detect malware that only reveals its malicious nature through execution, such as logic bombs triggered by specific conditions." This underscores the challenge that purely static file analysis faces against threats that are designed to remain invisible until specific circumstances activate them.

- **Zero-Day Exploits**

Are cyberattacks that target previously unknown software vulnerabilities—called "zero-day vulnerabilities"—for which no official patch, fix, or mitigation exists at the time of the attack. The term "zero-day" refers to the fact that developers have had zero days to address and patch the vulnerability since its discovery by attackers. Because these vulnerabilities are unknown to software vendors and security researchers, there are no signatures, patches, or specific defenses available to detect or block attacks exploiting them. Zero-day exploits are among the most valuable and dangerous tools in an attacker's arsenal, often sold on underground markets for hundreds of thousands or even millions of dollars.

They are frequently used in targeted attacks by advanced persistent threats (APTs), nation-state actors, and sophisticated cybercriminal organizations against high-value targets such as government agencies, defense contractors, financial institutions, and technology companies. The document explicitly identifies zero-day threats as a critical challenge for signature-based detection, stating that "signature-based methods in particular have failed to detect unknown threats" and that zero-day threats represent "periods of maximum vulnerability when traditional defenses are ineffective." This highlights why Trustfile's hybrid approach—combining signature-based detection, heuristic analysis, entropy analysis, and external threat intelligence—is necessary to provide defense-in-depth against both known and emerging threats.

- **Attack Surface**

Refers to the total sum of all possible points, vectors, or avenues through which an unauthorized user or malicious actor can attempt to enter, access, or extract data from a system, network, or organization. In cybersecurity, the attack surface encompasses every hardware, software, network connection, data repository, user account, API endpoint, cloud service, mobile device, IoT device, and human interaction that could potentially be exploited. The concept is rooted in the understanding that every feature, service, connection, or access point added to a system increases its complexity and, consequently, its potential vulnerability to

attacks. For example, when an organization enables remote work, adopts cloud storage, allows BYOD (bring your own device) policies, or integrates third-party APIs, it expands its attack surface by creating new entry points for attackers. The document uses the term "attack surface" in the context of digital file sharing, stating that "this same convenience has created a vast attack surface that malicious actors actively exploit to distribute harmful software." This refers to how the proliferation of file sharing channels—email, cloud storage, social media, USB drives, peer-to-peer networks—has exponentially increased the opportunities for malware distribution. Effective cybersecurity involves not just deploying defensive tools but also actively reducing the attack surface through measures such as network segmentation, access control, disabling unnecessary services, and minimizing the number of trusted file sources.

- **Social Engineering**

Psychological manipulation of people into performing actions or divulging confidential information, exploiting human psychology rather than technical hacking techniques to breach security. Unlike attacks that exploit software vulnerabilities, social engineering targets the "human element"—what security professionals often call the "weakest link in the security chain." Common social engineering techniques include phishing (sending fraudulent emails that appear to come from trusted sources), pretexting (creating a fabricated scenario to obtain information), baiting (offering something enticing to trick victims into compromising security), quid pro quo (offering a service or benefit in exchange for information), and tailgating (physically following authorized personnel into restricted areas).

In the context of file-based malware, social engineering is frequently used to trick users into opening infected attachments by making emails appear to come from colleagues, banks, shipping companies, or government agencies. The document emphasizes that "the human element represents the most significant vulnerability in cybersecurity frameworks" and that "a substantial majority of successful cyberattacks exploit human behavior rather than purely technical vulnerabilities." It describes how psychological factors—optimism bias (believing one won't be targeted), time pressure, lack of technical knowledge, and inherent trust in familiar file formats—make users susceptible to social engineering attacks. The Trustfile system addresses this by not only detecting malicious files but also educating users about threat indicators, helping them develop independent security judgment capabilities.

- **File Extension Spoofing and Double Extensions**

A deceptive technique used by malware distributors to disguise the true nature of a file by manipulating its visible filename and extension while the actual file type remains dangerous. In most operating systems, file extensions (the letters after the last dot in a filename, such as .exe, .pdf, or .jpg) determine how the system handles the file and what icon is displayed. Attackers exploit this by making executable files appear as harmless documents. Double extension techniques are a specific form of spoofing where filenames are constructed with two extensions, such as "document.pdf.exe" or "invoice.docx.scr". Many operating systems, by default, hide known file extensions, so users see only "document.pdf" or "invoice.docx" and believe they are opening safe document files, when in reality they are executing malicious programs. Some attackers also use Unicode characters, right-to-left override characters, or spaces to further obscure the true extension.

The document explicitly identifies these techniques as major challenges, stating that "file extension spoofing allows attackers to disguise executable files as documents by manipulating the visible filename while the actual file type remains executable" and that "double extension techniques present filenames such as 'document.pdf.exe' where the operating system hides the true extension." The Trustfile system counters these threats through file type verification that examines internal file structures and "magic numbers" (unique byte sequences at the beginning of files that identify their true format) rather than relying solely on visible filenames.

- **Magic Numbers**

Magic numbers (also called file signatures or magic bytes) are unique sequences of bytes at the beginning of a file that identify its format or type, regardless of its filename or extension. These byte sequences are standardized and recognized by operating systems, applications, and security tools to determine how a file should be processed. For example, a PDF file typically begins with the bytes %PDF (hex: 25 50 44 46), a JPEG image starts with FF D8 FF, a ZIP archive begins with 50 4B 03 04, and an executable file (PE format on Windows) starts with 4D 5A (the ASCII characters "MZ"). Magic numbers are embedded in the file header during creation and remain consistent across all files of that type.

Because they are part of the actual file content rather than metadata, magic numbers provide a reliable method for verifying a file's true type—unlike file extensions, which can be easily changed by users or attackers. The document mentions "magic numbers" in the context of Trustfile's file type verification capability, which "examines internal file structures and magic numbers to confirm that actual file types match their extensions and metadata claims." This is a critical defense against file extension spoofing, polyglot files, and other deception

techniques where attackers disguise malicious executables as harmless documents.

- **Hashing**

File hashing is a cryptographic process that generates a unique, fixed-length alphanumeric string—called a hash value or hash digest—from the contents of a file using a mathematical algorithm. Common hashing algorithms include MD5 (Message Digest 5), SHA-1 (Secure Hash Algorithm 1), and SHA-256 (Secure Hash Algorithm 256-bit). The fundamental property of a hash function is that even a minuscule change to the file's contents—such as adding a single space or changing one bit—will produce a completely different hash value. This makes hashes extremely useful for identifying files: if two files have identical hash values, they are virtually guaranteed to contain identical data. In malware detection, file hashing serves two primary purposes: (1) known malware identification, where the hash of a scanned file is compared against databases of hashes belonging to known malicious files, enabling instant identification of previously encountered threats; and (2) file integrity verification, where hashes are used to confirm that files have not been tampered with or corrupted. The document describes file hashing as a core component of Trustfile's detection architecture, stating that "file hashing generates unique cryptographic fingerprints that enable rapid identification of known malicious files through database comparison." The system also integrates with external API services like VirusTotal that maintain extensive hash databases, allowing Trustfile to identify emerging threats without needing to store massive local databases.

- **Defense-in-Depth**

Comprehensive cybersecurity strategy that employs multiple layers of security controls and defensive measures throughout an information technology system to protect data and resources. Rather than relying on a single security mechanism—which, if breached, leaves the entire system exposed—defense-in-depth creates redundant, overlapping layers of protection so that if one layer fails, others remain to thwart the attack. The concept is borrowed from military strategy, where defensive positions are arranged in multiple lines to slow and weaken an advancing enemy. In cybersecurity, these layers typically include: physical security (locks, guards, surveillance), network security (firewalls, intrusion detection systems, network segmentation), endpoint security (antivirus, host intrusion prevention), application security (secure coding, input validation), data security (encryption, access controls), and user awareness training (phishing education, security policies).

The document explicitly describes Trustfile's approach as providing "defense-in-depth against diverse threat categories" by integrating multiple complementary static analysis techniques—file hashing, entropy analysis, script detection, size anomaly detection, and file type verification—along with external API-based threat intelligence. Each technique addresses different threat vectors and compensates for the limitations of others: hashing catches known threats, entropy analysis detects packed/encrypted malware, script detection finds embedded malicious code, size analysis flags abnormal files, and type verification prevents spoofing. This multi-layered approach significantly improves the probability that at least one detection layer will identify a threat, even if others are evaded.

- **False Positives and False Negatives**

Two types of errors that occur in detection and classification systems, representing the fundamental trade-offs in security accuracy. A false positive occurs when a security system incorrectly flags a legitimate, harmless file or activity as malicious—essentially a "false alarm." For example, if an antivirus program incorrectly identifies a legitimate business document as a virus and quarantines it, that is a false positive. Excessive false positives are problematic because they disrupt normal operations, waste administrative resources investigating non-threats, erode user trust in the security system, and may lead users to disable security features out of frustration.

A false negative occurs when a security system fails to detect an actual threat, allowing malicious files or activities to pass through undetected. For example, if a new variant of ransomware slips past antivirus software because its signature is not yet in the database, that is a false negative. False negatives are particularly dangerous because they result in successful attacks, data breaches, and system compromises. The document discusses these concepts extensively in the context of Trustfile's classification system, noting that "minimizing false negatives (missed threats) while maintaining acceptable false positive rates" is a critical design goal.

The system uses multiple complementary detection techniques to reduce both types of errors: combining local heuristic analysis with external intelligence validation creates a hybrid model that balances speed and privacy with comprehensive coverage. Machine learning evaluation metrics such as precision (minimizing false positives), recall (minimizing false negatives), and F1-score (balancing both) are used to measure and optimize classification performance.

- **Threat Intelligence**

Threat intelligence (also called cyber threat intelligence or CTI) is evidence-based knowledge about existing or emerging threats to an organization's assets, including information about threat actors, their tactics, techniques, procedures (TTPs), indicators of compromise (IOCs), malware families, vulnerabilities, and attack campaigns. This intelligence is collected, analyzed, and disseminated to help organizations understand the threat landscape, anticipate attacks, and make informed security decisions.

Threat intelligence can be categorized by its intended audience and application: strategic intelligence provides high-level insights about threat trends, actor motivations, and geopolitical context for executive decision-makers; tactical intelligence details specific TTPs and attack methods for security operations teams; operational intelligence provides real-time information about active threats, IOCs, and campaign indicators for immediate defensive action; and technical intelligence includes specific data such as malware signatures, IP addresses, domain names, and file hashes for automated security tools.

The document describes how Trustfile integrates with "external API-based malware intelligence services" that "maintain extensive databases of known malware signatures, behavioral indicators, and reputation scores accumulated from global threat intelligence networks." These services leverage economies of scale and distributed observation that individual organizations cannot replicate, enabling Trustfile to identify emerging threats that may not yet be represented in local databases. The combination of local heuristic analysis and external intelligence validation creates a hybrid detection model that balances the speed and privacy of local processing with the comprehensive coverage of cloud-based threat intelligence.

- **Data Exfiltration**

Data exfiltration (also called data extrusion or data theft) is the unauthorized transfer of data from within an organization to an external destination or attacker-controlled location. It is a critical objective of many cyberattacks, particularly those carried out by advanced persistent threats (APTs), insider threats, and organized cybercriminal groups. Data exfiltration can occur through various channels: network-based methods (uploading data to external servers via HTTP, FTP, DNS tunneling, or cloud storage services), physical methods (copying data to USB drives, external hard drives, or printed documents), email-based methods (sending sensitive attachments to external addresses), and application-based methods (using legitimate cloud services or file-sharing platforms to transfer data).

The types of data targeted for exfiltration include intellectual property, trade secrets, customer databases, financial records, personal identifiable information (PII), credentials, and strategic business plans. The document mentions data exfiltration in the context of ransomware attacks, noting that "attackers often exfiltrate the data in the process" of encrypting files, adding an additional layer of extortion where victims are threatened with public disclosure of stolen data if they refuse to pay the ransom. This "double extortion" tactic has become increasingly common in modern ransomware campaigns. The Trustfile system's pre-access scanning approach aims to prevent data exfiltration by intercepting malicious files before they can execute and establish communication channels with attacker-controlled servers.

- **Black Box**

A term used in cybersecurity and software testing to describe a system, process, or detection mechanism whose internal workings are hidden, opaque, or not visible to users or external observers. In the context of security software, a "black box" approach means that the system consumes inputs (such as files to be scanned) and produces outputs (such as "threat detected" or "file clean") without revealing the analytical reasoning, specific indicators examined, or confidence levels behind its decisions. The document uses this term critically, stating that "the 'black box' nature of automated remediation prevents users from understanding the rationale behind false positives, potentially leading to frustration and security software disablement."

The authors argue that traditional antivirus software operates as a black box: when it detects a threat, it typically quarantines or deletes the file automatically, presenting users with minimal information beyond a generic threat name. This opacity creates several negative outcomes: users cannot verify the accuracy of detections, cannot learn to recognize similar threats independently, may distrust the software when false positives occur, and remain perpetually dependent on automated protection without developing their own security awareness. Trustfile is explicitly designed to counter this black box approach by providing "detailed explanations of the specific indicators that triggered threat classifications, the technical significance of these indicators, and actionable guidance for users." This transparency transforms each scanning encounter into a learning opportunity, gradually building user knowledge and threat awareness while maintaining security effectiveness.

CHAPTER 2

REVIEW OF RELATED LITERATURE AND STUDIES

Foreign Studies

According to the recent studies, malicious software (malware) is increasing at an alarming rate, and some malware can hide in the system by using different obfuscation techniques. In order to protect computer systems and the Internet from the malware, the malware needs to be detected before it affects a large number of systems. Recently, there have been made several studies on malware detection approaches. However, the detection of malware still remains problematic. Signature-based and heuristic-based detection approaches are fast and efficient to detect known malware, but especially signature-based detection approach has failed to detect unknown malware. On the other hand, behavior-based, model checking-based, and cloud-based approaches perform well for unknown and complicated malware; and deep learning-based, mobile devices-based, and IoT-based approaches also emerge to detect some portion of known and unknown malware. However, no approach can detect all malware in the wild. This shows that to build an effective method to detect malware is a very challenging task, and there is a huge gap for new studies and methods. This paper presents a detailed review on malware detection approaches and recent detection methods which use these approaches. Paper goal is to help researchers to have a general idea of the malware detection approaches, pros and cons of each detection approach, and methods that are used in these approaches.

To protect legitimate users and companies from malware, malware need to be detected. Malware detection is the process of determining whether a given program has malicious intent or not. In early days, signature-based detection approach was used widely to detect malware. However, this approach has some limitations such as it cannot detect unknown and new generation malware. In process of time, researchers proposed new approaches including behavioral-, heuristic-, and model checking-based detection. With these approaches, datamining and machine learning (ML) algorithms are also started to be used widely in malware detection. Recently, new approaches have been proposed such as deep learning-, cloud-, mobile devices-, and IoT-based detection. For known and some of unknown malware, heuristic detection approach performs well. On the other hand, for unknown and complicated malware; behavior-, model checking-, and cloud-based approaches perform better. Deep learning-, mobile devices-, and IoT-based approaches also emerge to detect some portion of known and unknown malware. It has not been proved exactly that one detection approach is more effective than the others. This is because each method has its own advantages and disadvantages, and in different situation one method can detect better than another. Even though several new methods have been proposed by using different malware detection approaches, no method could detect all new generation and sophisticated malware. This shows that building an effective

method to detect malware is a very challenging task, and there is a huge demand for new studies and methods.

Aslan and Samet (2020) conducted a comprehensive review of malware detection approaches published in IEEE Access, examining how malicious software continues to grow at an alarming rate and frequently conceals itself within systems through various obfuscation techniques. Their study found that while signature-based and heuristic-based detection approaches are fast and efficient for identifying known malware, signature-based methods in particular have failed to detect unknown threats. In contrast, behavior-based, model checking-based, and cloud-based approaches perform well against unknown and more complex malware, and emerging methods such as deep learning-based, mobile device-based, and IoT-based approaches also show promise in detecting both known and unknown variants.

The study's examination of malware growth rates quantifies the expanding threat environment that motivates Trustfile's development. Industry reports consistently document exponential increases in new malware samples, with millions of new variants detected annually. This growth results from multiple factors: the low cost and high profitability of malware development, the availability of malware-as-a-service platforms that democratize attack capabilities, the automation of malware generation through polymorphic and metamorphic engines, and the expanding attack surface created by IoT devices, cloud services, and remote work arrangements. The "alarming rate" characterization reflects not merely numerical growth but increasing sophistication, evasion capabilities, and impact severity.

The focus on obfuscation techniques addresses a critical challenge for detection systems. Malware developers employ diverse obfuscation strategies to evade signature-based detection, including code encryption, packing, polymorphism, metamorphism, anti-debugging, anti-virtualization, and steganography. These techniques transform malware appearance between generations or during execution while preserving malicious functionality. Aslan and Samet's examination of these techniques provides essential context for understanding why signature-based methods struggle and why alternative approaches are necessary.

The comparative findings regarding detection methodologies inform Trustfile's hybrid architecture design. Signature-based detection, while fast and efficient for known threats, exhibits fundamental limitations against novel malware that lacks matching signatures in detection databases. The temporal gap between malware emergence and signature creation—sometimes hours or days—creates windows of vulnerability. Heuristic-based detection improves upon signatures by identifying suspicious patterns and behaviors, but basic heuristics can be circumvented by sophisticated obfuscation and may generate false positives that erode user trust.

The superior performance of behavior-based approaches reflects their focus on what malware does rather than what it looks like. By monitoring runtime behaviors—file system modifications, registry changes, network communications, process injections—behavior-based systems can identify malicious activity even from previously unseen malware variants. However, behavior-based detection requires execution environments that introduce risks, performance impacts, and complexity.

Model checking-based approaches apply formal verification techniques to analyze program behaviors against security specifications, identifying violations that indicate malicious intent. These methods offer strong theoretical foundations but face scalability challenges with complex software. Cloud-based approaches leverage distributed computing resources and collective intelligence to analyze files at scale, correlating observations across large user populations to identify emerging threats.

The emerging methods identified by Aslan and Samet—deep learning-based, mobile device-based, and IoT-based approaches—represent the cutting edge of malware detection research. Deep learning techniques, including convolutional neural networks, recurrent neural networks, and autoencoders, can automatically learn complex feature representations from raw data, potentially capturing subtle malware indicators that elude manual feature engineering. Mobile device-based approaches address the unique constraints and threat landscape of smartphone and tablet platforms. IoT-based approaches tackle the expanding attack surface of connected devices with limited computational resources and diverse architectures.

The study's conclusion that these emerging methods show promise for detecting both known and unknown variants directly supports Trustfile's incorporation of advanced detection techniques alongside traditional methods. The hybrid approach advocated by the literature—combining signature, heuristic, behavioral, and intelligence-based detection—maximizes coverage across threat categories while mitigating the limitations of any single methodology.

Local Studies

Ransomware is a type of malicious software – Malware- that persists and evolves accordingly to organizational countermeasures. It aims to block user access to a system and its data, typically for monetary gain. Two major methods of doing so are the blatant blocking of a system's use and the encryption of these data. This poses a significant threat to organizations, causing major financial losses and disclosure of confidential or sensitive data to unauthorized individuals, as attackers often exfiltrate the data in the process. In the Philippines, the general overview of the cybersecurity landscape has been explored and evaluated. It is observed that the country is particularly lacking in its investment in the field, with a misconception of its insignificance,

despite its looming and ever-growing threats. This has caused major data leakages and compromised services in recent years. Despite these existing evaluations, there remains an opportunity to delve deeper into this for a richer understanding. This journal intends to provide the initial groundworks for this, by assessing Ransomware experiences and impacts in Philippine organizations of different sectors and correlating such with existing knowledge. This is particularly conducted by thoroughly evaluating the qualitative data and extracting its key features for its automated categorization. Aaron Lensmer Abdon. (2024).

Ransomware is a type of malicious software – Malware- that persists and evolves according to organizational countermeasures. It aims to block user access to a system and its data, typically for monetary gain. Two major methods of doing so are the blatant blocking of a system's use and the encryption of these data. This poses a significant threat to organizations, causing major financial losses and disclosure of confidential or sensitive data to unauthorized individuals, as attackers often exfiltrate the data in the process. In the Philippines, the general overview of the cybersecurity landscape has been explored and evaluated. It is observed that the country is particularly lacking in its investment in the field, with a misconception of its insignificance, despite its looming and ever-growing threats. This has caused major data leakages and compromised services in recent years. Despite these existing evaluations, there remains an opportunity to delve deeper into this for a richer understanding.

The focus on ransomware reflects this malware category's particular relevance to Philippine organizations and its alignment with Trustfile's protective objectives. Ransomware has emerged as one of the most financially devastating cyber threats globally, with attacks increasing in frequency, sophistication, and impact. The persistence and evolution characteristics described in the local study highlight the adaptive nature of ransomware operators, who continuously refine their techniques in response to defensive measures. This arms race necessitates equally adaptive detection and prevention capabilities.

The dual attack mechanisms—system blocking and data encryption—represent the operational modalities through which ransomware achieves its objectives. System blocking prevents users from accessing computing resources, displaying ransom demands and payment instructions. Data encryption renders files unreadable without decryption keys held by attackers, creating strong incentives for ransom payment. The combination of these mechanisms maximizes pressure on victims while demonstrating attacker capability.

The organizational impact extends beyond immediate financial losses from ransom payments to encompass operational disruption, reputational damage, regulatory penalties, and long-term recovery costs. Business interruption during ransomware incidents can last days or weeks, with critical systems offline and manual workarounds implemented. Data exfiltration by attackers—stealing sensitive information before encryption—adds extortion leverage through threats of public disclosure, transforming ransomware into data breach incidents with regulatory implications.

The Philippine cybersecurity landscape assessment reveals systemic challenges that amplify ransomware and broader malware threats. The identified investment deficiency reflects both public and private sector underfunding of cybersecurity infrastructure, personnel, training, and research. This underinvestment manifests in outdated security technologies, insufficient security staffing, inadequate incident response capabilities, and limited threat intelligence resources. The "misconception of insignificance" suggests a cultural or leadership-level failure to recognize cybersecurity as a strategic priority, potentially viewing it as a cost center rather than a business enabler or risk management necessity.

The consequences of this underinvestment are tangible and severe. Major data leakages compromise citizen privacy, organizational secrets, and national security information. Compromised services disrupt government operations, business transactions, healthcare delivery, and educational activities. These incidents damage public trust in digital services, potentially slowing digital transformation initiatives that could otherwise drive economic and social development.

The identified opportunity for deeper investigation supports the research motivation for Trustfile. By developing and evaluating a practical file security solution within the Philippine context, this study contributes to addressing the documented cybersecurity gaps. The system's focus on pre-access scanning, user education, and accessible design specifically addresses the resource constraints and knowledge deficits identified in the local landscape. The academic research framework ensures rigorous evaluation and documentation, creating knowledge resources that can inform broader cybersecurity improvement efforts.

Related Literature

Collective intelligence systems like Chat Generative Pre-Trained Transformer (ChatGPT) have emerged. They have brought both promise and peril to cybersecurity and privacy protection. This study introduces novel approaches to harness the power of artificial intelligence (AI) and big data analytics to enhance security and privacy in this new era. Contributions could explore topics such as: leveraging natural language processing (NLP) in ChatGPT-like systems to strengthen information security; evaluating privacy-enhancing technologies to maximize data utility while minimizing personal data exposure; modeling human behavior and agency to build secure and ethical human-centric systems; applying machine learning to detect threats and vulnerabilities in a data-driven manner; using analytics to preserve privacy in large datasets while enabling value creation; crafting AI techniques that operate in a trustworthy and explainable manner. This article advances the state-of-the-art at the intersection of cybersecurity, privacy, human factors, ethics, and cutting-edge AI, providing impactful solutions to emerging challenges. Our research presents a revolutionary approach to malware detection that leverages deep learning (DL) based methodologies to automatically learn features from raw

data. Our approach involves constructing a grayscale image from a malware file and extracting features to minimize its size. This process affords us the ability to discern patterns that might remain hidden from other techniques, enabling us to utilize convolutional neural networks (CNNs) to learn from these grayscale images and a stacking ensemble to classify malware. The goal is to model a highly complex nonlinear function with parameters that can be optimized to achieve superior performance. To test our approach, we ran it on over 6,414 malware variants and 2,050 benign files from the Mallmg collection, resulting in an impressive 99.86 percent validation accuracy for malware detection. Furthermore, we conducted a classification experiment on 15 malware families and 13 tests with varying parameters to compare our model to other comparable research. Our model outperformed most of the similar research with detection accuracy ranging from 47.07% to 99.81% and a significant increase in detection performance. Our results demonstrate the efficacy of our approach, which unlocks the hidden patterns that underlie complex systems, advancing the frontiers of computational security. Naeem MR, Amin R, Farhan M, Alotaibi FA, Alnfai MM, Sampedro GA, Karovič V.

Our research presents a revolutionary approach to malware detection that leverages deep learning (DL) based methodologies to automatically learn features from raw data. Our approach involves constructing a grayscale image from a malware file and extracting features to minimize its size. To test our approach, we ran it on over 6,414 malware variants and 2,050 benign files from the Mallmg collection, resulting in an impressive 99.86 percent validation accuracy for malware detection (Caluza, 2024).

The featured research by Caluza (2024) demonstrates a novel application of deep learning to malware detection through image representation techniques. This approach transforms the malware detection problem from binary analysis into image classification, leveraging the powerful capabilities of convolutional neural networks (CNNs) developed for computer vision tasks.

The grayscale image construction process involves interpreting binary file contents as pixel intensity values, creating two-dimensional image representations that encode structural and statistical features of the original files. This transformation is non-trivial and requires careful consideration of image dimensions, byte-to-pixel mapping strategies, and normalization techniques. Common approaches include reading file bytes sequentially to fill image rows, with image width determined by file size or fixed parameters. The resulting images capture patterns including code sections, data segments, padding, and structural regularities that may distinguish malware from benign files.

Feature extraction through deep learning automates the identification of discriminative patterns without manual feature engineering. Traditional machine learning approaches require security experts to identify and quantify relevant features—byte frequencies, instruction sequences, API calls, string patterns—creating features that may miss novel indicators or incorporate researcher biases. Deep learning models learn hierarchical feature representations directly from raw image data, potentially capturing subtle patterns invisible to human analysts.

The dataset scale—6,414 malware variants and 2,050 benign files from the Mallmg collection—provides substantial training and validation material. The Mallmg dataset is a well-known benchmark in malware image analysis research, containing grayscale images of malware files categorized by family. This dataset enables reproducible research and comparative evaluation across studies. The class distribution, with malware outnumbering benign files, reflects real-world prevalence while requiring careful handling to prevent class imbalance from biasing model training.

The reported 99.86 percent validation accuracy represents exceptional performance, approaching the theoretical maximum for binary classification tasks. This accuracy metric, calculated on held-out validation data not used during training, indicates strong generalization capability—the model's ability to correctly classify new, unseen files. However, accuracy alone provides incomplete performance assessment, as it can be misleading in imbalanced datasets or when false negative costs exceed false positive costs. Comprehensive evaluation would additionally report precision, recall, F1-score, and area under the ROC curve.

The implications for Trustfile are significant. While Trustfile's current scope focuses on static analysis techniques including hashing, entropy analysis, and script detection, the demonstrated effectiveness of deep learning image analysis suggests a promising enhancement path. Future iterations could incorporate CNN-based classification as an additional detection layer, potentially improving accuracy against novel threats. The image representation approach also offers intuitive visualization capabilities, allowing users to see visual differences between safe and suspicious files.

The Caluza (2024) research validates the broader research direction of applying advanced analytical techniques to malware detection, supporting Trustfile's hybrid methodology that combines multiple detection paradigms. The high accuracy achieved demonstrates that sophisticated detection is feasible and effective, motivating continued investment in advanced detection capabilities.

Synthesis of the Reviewed Literature

The reviewed literature and studies collectively highlight a significant paradigm shift in malware detection, moving from traditional, reactive methods toward proactive, data-driven intelligence. The synthesis section integrates findings from the reviewed literature into coherent themes that inform and justify Trustfile's research approach. This integrative analysis identifies convergent findings, complementary insights, and overarching trends that establish the theoretical foundation for the proposed system.

The identified paradigm shift from reactive to proactive detection represents a fundamental evolution in cybersecurity philosophy. Traditional reactive methods respond to known threats after they have been identified, analyzed, and documented. Signature-based detection epitomizes this approach, maintaining databases of known malware fingerprints and blocking matches. While effective against established threats, reactive methods are inherently backward-looking, unable to address novel attacks until they have been discovered, analyzed, and added to detection databases. The proactive paradigm, in contrast, anticipates and prevents threats through behavioral analysis, anomaly detection, threat intelligence, and predictive modeling. This shift reflects the recognition that the speed of modern attacks—particularly automated threats like worms and ransomware—outpaces reactive response capabilities.

The data-driven intelligence aspect emphasizes the increasing reliance on large-scale data analysis, machine learning, and artificial intelligence to identify patterns and predict threats. Rather than relying solely on expert-defined rules and signatures, data-driven approaches analyze vast datasets of file characteristics, network behaviors, and system events to discover subtle indicators of compromise. This approach scales effectively with data volume and computational resources, improving detection capabilities as more data becomes available. Aslan and Samet (2020) establish the foundational problem: the inadequacy of traditional signature-based methods in the face of modern, "zero-day" threats. This global challenge is mirrored in the local studies by Caluza and Valenzuela (2021), who emphasize that protecting data integrity in the Philippines requires moving beyond standard scanning.

The synthesis draws explicit connections between foreign and local research, demonstrating that the identified challenges are neither unique to specific regions nor adequately addressed by existing solutions. Aslan and Samet's documentation of signature-based inadequacy against zero-day threats—previously unknown vulnerabilities and malware variants—establishes the technical imperative for alternative detection approaches. Zero-day threats exploit vulnerabilities before patches are available and distribute malware before signatures are created, representing periods of maximum vulnerability when traditional defenses are ineffective.

The local studies' emphasis on data integrity protection reflects the practical consequences of detection inadequacy. When standard scanning fails to detect advanced threats, organizations suffer data breaches, system compromises, and operational disruptions that undermine data integrity—the assurance that information is accurate, complete, and unaltered. The Philippine context adds urgency through documented underinvestment and resulting vulnerability, suggesting that innovative, cost-effective solutions like Trustfile are particularly needed in resource-constrained environments.

The reference to Caluza and Valenzuela (2021) extends the local research context, indicating a body of Philippine cybersecurity research that informs and supports the current study. This research continuity demonstrates that Trustfile contributes to an ongoing scholarly conversation rather than operating in isolation.

A central theme in this synthesis is the use of Convolutional Neural Networks (CNN) and Grayscale Image Representation. While Aslan and Samet identify deep learning as a "promising" emerging method, the local work of Caluza (2024) demonstrates its practical high-level efficacy, achieving a 99.86% accuracy rate. By converting binary files into visual data, these researchers overcome the "obfuscation" issues mentioned in the foreign literature. Ultimately, these studies provide a strong justification for the current research's focus on utilizing advanced AI-driven visual analysis as a superior alternative to traditional file scanning.

The CNN and grayscale image representation theme represents the technical convergence point between reviewed literature and Trustfile's potential enhancement trajectory. Convolutional Neural Networks, originally developed for image recognition tasks, have proven remarkably effective at identifying visual patterns including textures, shapes, and structural regularities. When applied to malware images, CNNs can learn to distinguish malicious from benign files based on visual patterns that encode binary structure, code organization, and data distribution. The practical validation of deep learning's promise bridges the gap between theoretical potential and empirical demonstration. Aslan and Samet's identification of deep learning as "promising" establishes the conceptual foundation, while Caluza's 99.86% accuracy provides concrete evidence of effectiveness. This progression from promise to proof supports investment in deep learning integration as a high-confidence enhancement strategy.

The obfuscation overcoming mechanism is particularly significant. Traditional static analysis struggles with packed, encrypted, and polymorphic malware because the actual malicious code is hidden until runtime. However, the packing and encryption processes themselves leave traces in file structure and byte distribution that may be visible in image representations. Packing typically creates distinctive patterns in the image including high-entropy regions, structural boundaries between packed and unpacked sections, and characteristic headers. These visual indicators may persist even when the underlying code is transformed, enabling detection that signature-based methods cannot achieve.

The synthesis concludes by establishing the justification for Trustfile's research direction. The combination of documented detection inadequacy, demonstrated alternative effectiveness, and specific regional needs creates a compelling case for developing and evaluating advanced file security solutions. While Trustfile's initial implementation focuses on static analysis techniques, the reviewed literature provides strong justification for progressive enhancement incorporating AI-driven visual analysis as the system matures.

CHAPTER 3

METHODOLOGY

Research Design

This study uses a developmental research design. It focuses on designing, developing, and testing the TrustFile system as a web-based file security platform.

The research design section establishes the overarching methodological framework guiding the study. Developmental research design, also known as Research and Development (R&D) methodology, is particularly appropriate for creating and evaluating technological innovations. This design emphasizes the systematic process of identifying needs, designing solutions, developing prototypes, and testing effectiveness within real or simulated contexts. Unlike purely descriptive or explanatory research that seeks to understand existing phenomena, developmental research actively creates new artifacts—in this case, a software system—and evaluates their utility.

The focus on design encompasses architectural planning, user interface specification, algorithm selection, and integration strategy. Design activities transform abstract requirements into concrete technical specifications that guide implementation. The design process involves trade-off analysis balancing competing objectives: security versus usability, comprehensiveness versus performance, automation versus user control, local processing versus external dependency. Design decisions are documented to enable evaluation, replication, and future modification.

The development component involves the actual construction of the system according to specifications. This includes coding software components, configuring infrastructure, integrating third-party services, and assembling the complete system. Development follows established software engineering practices including version control, code review, testing, and documentation. The developmental nature of the research means that the system artifact itself constitutes a primary research output, with its characteristics and capabilities directly addressing the research objectives.

Testing validates that the developed system meets requirements and functions as intended. Testing encompasses multiple levels from individual component verification through integrated system evaluation to user acceptance confirmation. Test results provide empirical evidence of system effectiveness, identifying strengths to leverage and deficiencies to address. The testing phase generates data that supports conclusions about system viability and guides refinement decisions.

The web-based platform specification constrains the design space while enabling specific capabilities. Web architectures enable universal access, simplified deployment, centralized management, and cross-platform compatibility. However, they also impose limitations including browser security restrictions, network dependency, and performance constraints that must be addressed through careful design.

The research is divided into the following phases:

1. **Descriptive Phase** - In this phase, the researchers studied common problems related to file security. This includes the risks of opening unscanned files and the lack of tools for checking files before access. The researchers also identified the needed features of the system based on these problems. These include file scanning, threat detection, and displaying clear results to users.

The descriptive phase establishes the problem foundation upon which the subsequent development is built. This phase employs descriptive research methods to characterize the current state of file security practices, identify deficiencies, and define requirements. Descriptive research answers "what is" questions through observation, survey, case study, and literature review, providing factual grounding for solution design.

The study of common problems involves systematic investigation of file security challenges encountered by target users. This investigation may include direct observation of user file handling behaviors, analysis of security incident reports, review of help desk tickets related to malware infections, interviews with IT administrators, and survey of user security knowledge and practices. The goal is to develop comprehensive understanding of the specific problems that Trustfile should address, grounded in real-world experience rather than theoretical assumptions.

The risks of opening unscanned files encompass the full spectrum of potential consequences from malware execution. These risks include immediate system compromise, data theft or destruction, ransomware encryption, credential harvesting, network propagation, and persistent attacker access. Quantifying these risks involves analyzing incident data, threat intelligence reports, and cost studies that document the frequency and impact of file-based attacks

2. **Developmental Phase** - In this phase, the researchers designed and developed the system. This includes creating the file scanning process, implementing the file analysis method, such as hashing, entropy analysis, pattern detection, and file type checking, and integrating an API for malware detection. All components were combined to create a working prototype of the system.

The developmental phase represents the core creative activity of the research, transforming requirements into a functional software system through systematic design and implementation processes. This phase is where the theoretical understanding developed in the descriptive phase is translated into practical technical solutions.

Creating the file scanning process involves designing the end-to-end workflow from user file submission through analysis completion to result presentation. This includes defining supported file types and size limits, designing secure upload mechanisms, specifying scan queue management, determining analysis sequencing, and designing result formatting. The scanning process must balance thoroughness with efficiency, comprehensiveness with usability, and security with accessibility.

Implementing file analysis methods requires developing software modules for each detection technique. Hashing implementation involves selecting appropriate hash algorithms (MD5, SHA-256), computing file digests, and comparing against threat databases. Entropy analysis implementation involves developing algorithms for Shannon entropy calculation, establishing baseline profiles for common file types, and defining thresholds for anomaly flagging. Pattern detection involves creating regular expressions, byte sequence signatures, and heuristic rules for identifying suspicious content. File type checking involves implementing magic number verification, structure validation, and extension comparison.

API integration for malware detection involves selecting appropriate external services, implementing secure API clients, designing query strategies, handling responses, and integrating results into the overall threat assessment. This includes managing API keys, implementing rate limiting, designing caching strategies, and creating fallback mechanisms for service unavailability.

The working prototype represents the tangible output of the developmental phase—a functional system that demonstrates core capabilities, validates technical approaches, and provides a platform for testing and evaluation. The prototype need not include all envisioned features but must implement sufficient functionality to validate the research concept and enable meaningful evaluation.

3. **Experimental Phase** - In this phase, the system was tested using different types of files, such as normal files, suspicious files, and safe test malware like the EICAR Test file. The purpose of testing is to check if the system can properly detect and classify threats. The researchers also evaluated the accuracy of results and the overall performance of the system.

The experimental phase subjects the developed prototype to systematic evaluation under controlled conditions, generating empirical evidence of system capabilities and limitations. This phase applies scientific methodology to software evaluation, ensuring

that conclusions are supported by observable, measurable data rather than subjective impressions.

Testing with different file types ensures comprehensive evaluation across the spectrum of files that the system will encounter in production use. Normal files—legitimate documents, images, executables, and archives—test the system's ability to avoid false positives and correctly classify benign content. Suspicious files—files with anomalous but not definitively malicious characteristics—test the system's handling of uncertain cases and its classification granularity. Safe test malware like the EICAR Test file provides a standardized, harmless mechanism for validating malware detection capabilities without introducing actual threats into the testing environment.

The purpose of testing extends beyond simple pass/fail determination to comprehensive capability assessment. Proper detection validation confirms that the system identifies known threats, suspicious characteristics, and benign files appropriately. Classification validation confirms that assigned threat levels correspond accurately to actual risk. Accuracy evaluation quantifies detection precision through metrics including true positive rate, false positive rate, true negative rate, and false negative rate. Performance evaluation measures system responsiveness, throughput, resource consumption, and scalability under varying load conditions.

System Development Methodology

This study used the Agile Development Model in developing the System. This approach allows the researchers to improve the System step-by-step while continuously testing and making adjustments. It is useful because changes can easily be applied during the development process.

The Agile Development Model was selected for this research based on its alignment with the exploratory, iterative nature of academic software development. Agile methodologies, formalized in the Agile Manifesto of 2001, prioritize individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. These priorities resonate with research contexts where requirements evolve as understanding deepens, where working prototypes demonstrate progress more effectively than specifications, and where flexibility enables adaptation to unexpected challenges.

The step-by-step improvement characteristic of Agile involves breaking development into small, manageable increments called sprints or iterations, typically lasting one to four weeks. Each iteration delivers a potentially shippable product increment that demonstrates tangible progress and can be evaluated by stakeholders. This incremental approach reduces risk by identifying problems early, enables course correction based on feedback, and maintains stakeholder engagement through visible progress.

Continuous testing and adjustment represents a core Agile practice that integrates quality assurance throughout development rather than deferring it to a final testing phase. Each iteration includes testing activities that validate new functionality, regression testing that confirms existing functionality remains intact, and review meetings that assess progress and plan adjustments. This continuous validation ensures that defects are identified and corrected promptly, reducing the cost and complexity of late-stage fixes.

1. **Requirements Analysis** - In this phase, the researchers identified the common problems related to file security. Many users do not scan files before opening them, which can lead to possible malware infection. The researchers listed the needed features of the system, such as file upload scanning, local file scanning, and displaying results. The goal is to create a simple and easy-to-use system.

The requirements analysis phase establishes the foundation for all subsequent development activities by systematically identifying, documenting, and prioritizing what the system must accomplish. Effective requirements analysis ensures that development efforts address real user needs rather than assumed problems, reducing the risk of building a solution that fails to solve the intended problem.

Identifying common problems involves gathering information from multiple sources including user interviews, workplace observations, security incident reports, literature review, and analysis of existing systems. The specific problem of users not scanning files before opening reflects both behavioral patterns and systemic deficiencies. Behavioral factors include lack of awareness, time pressure, trust in file sources, and underestimation of risks. Systemic factors include absence of accessible scanning tools, complexity of existing security software, and integration gaps between security and workflow tools.

2. **System Design** - In this phase, the researchers planned how the system would function. The design includes the process of scanning files using methods such as hashing, entropy analysis, pattern detection, and file type checking. The researchers also planned the use of an API for malware detection. The interface was designed to be simple so users can easily upload and scan files.

The scanning process design specifies how files are analyzed from submission through classification. Hashing design involves selecting hash algorithms, defining comparison databases, and specifying match handling. Entropy analysis design involves defining calculation methods, establishing baseline profiles for file types, and setting anomaly thresholds. Pattern detection design involves creating signature databases, heuristic rules, and behavioral indicators. File type checking design involves implementing magic number verification, structure validation, and spoofing detection.

API integration design specifies how external malware intelligence services are incorporated. This includes selecting appropriate services based on coverage, accuracy, cost, and reliability; designing query formats and protocols; specifying response handling and result integration; and defining fallback behaviors for service unavailability. The API design must balance comprehensive threat intelligence with performance, privacy, and cost constraints.

Interface design focuses on user interaction, creating wireframes, mockups, and prototypes that visualize the user experience. Simplicity principles guide interface decisions: minimal required actions, clear visual hierarchy, intuitive navigation, consistent terminology, and helpful feedback. The upload interface enables easy file selection and submission. The scanning interface provides progress indication and status updates. The results interface presents findings clearly with appropriate detail levels for different user types.

3. **Development** - In this phase, the system was built step-by-step. The researchers first created the file upload and scanning features. After that, the file analysis methods, such as hashing, entropy, and pattern detection of malicious files. The system was able to classify files into different threat levels like Low, Medium, High, and Critical.

Step-by-step construction follows the Agile principle of incremental development, where functionality is built and integrated in small, manageable pieces rather than attempting complete implementation in a single effort. This approach enables early validation, reduces integration complexity, and maintains development momentum through visible progress.

The file upload feature implementation involves creating server-side endpoints that receive file submissions, validate submissions against size and type constraints, store files securely for analysis, and manage upload sessions. Client-side implementation provides the user interface for file selection, drag-and-drop functionality, progress indication, and error handling. Security considerations include protection against upload-based attacks, secure temporary storage, and access controls.

The scanning feature implementation orchestrates the analysis workflow, managing file queues, invoking analysis modules, aggregating results, and generating reports. This involves creating a scanning engine that coordinates multiple detection techniques, applies classification logic, and formats output for presentation.

File analysis method implementation develops the core detection capabilities. Hashing implementation computes cryptographic digests and compares against threat databases. Entropy implementation calculates statistical randomness measures and evaluates against baselines. Pattern detection implementation applies signatures and heuristics to identify suspicious content. Each method is implemented as a modular component that can be tested, maintained, and enhanced independently.

Classification implementation applies decision logic to combine individual method results into overall threat assessments. The Low, Medium, High, and Critical levels are defined with specific criteria, and classification algorithms assign files to appropriate levels based on detected indicators. This implementation includes threshold calibration, weight assignment for different indicators, and rule definition for classification boundaries.

4. **Testing** - The system was tested using different sample files to check if it works properly. The researchers used safe testing files such as the EICAR Test File to simulate malware detection without causing harm. This helped check if the system can detect and classify suspicious files correctly. Some errors were found during testing and were fixed to improve the system.

Testing with different sample files ensures comprehensive coverage of the file types and threat categories the system will encounter. Test files include legitimate documents, images, executables, and archives to validate benign file handling; files with suspicious but non-malicious characteristics to validate classification granularity; and safe test malware to validate malicious file detection without introducing actual threats.

The EICAR Test File provides a standardized, safe mechanism for validating malware detection. As a file containing only the EICAR test string, it triggers detection by security software without any malicious capability. This enables testing in production-like environments without risk of infection or spread. EICAR testing validates that the detection pipeline functions correctly from file submission through alert generation.

Detection and classification validation confirms that the system correctly identifies threats and assigns appropriate risk levels. This involves verifying that known malicious files are detected, that benign files are not falsely flagged, and that suspicious files receive appropriate intermediate classifications. Validation uses test datasets with known labels to quantify accuracy metrics.

Error identification and correction represents the iterative improvement process central to Agile development. Testing reveals defects, performance issues, usability problems, and security vulnerabilities that were not apparent during development. Each identified issue is documented, prioritized, corrected, and re-tested to ensure resolution. This continuous improvement cycle progressively enhances system quality and reliability.

5. **Deployment (Prototype)** - After testing, the system was presented as a working prototype. It shows how the system can scan and analyze files before opening them and display the result to users. The system can help users become more aware and careful when handling files.

The working prototype represents a functional system that implements essential features sufficiently to demonstrate viability and enable evaluation. While not necessarily production-ready, the prototype proves that the research concept can be realized in software and provides a platform for user feedback, performance assessment, and concept refinement.

Demonstration of core capabilities includes showing how files are uploaded, how scanning is performed, how results are presented, and how users interact with the system. This demonstration validates that the system addresses the problems identified in requirements analysis and achieves the objectives established in the research design.

The educational impact of the prototype extends beyond technical demonstration to user awareness enhancement. By showing users how files are analyzed, what indicators are examined, and how threats are identified, the system builds understanding that translates into more cautious file handling practices. This awareness enhancement represents a significant contribution to organizational security culture, complementing the technical protection provided by automated detection.

System Architecture

Trustfile works like a security checkpoint for files before users open them. The System has a simple web interface where users can upload a field web interface where users can upload files from their computer to scan. It is designed to be easy to use, so even users without technical knowledge can still use it properly.

The system architecture section describes the high-level structure and operational model of Trustfile, explaining how components interact to deliver file security services. Architecture description provides stakeholders with conceptual understanding of system operation without requiring detailed technical knowledge.

The security checkpoint analogy effectively communicates Trustfile's core function—intercepting files before they reach users, examining them for threats, and either permitting or blocking passage based on examination results. Like physical security checkpoints that inspect luggage, verify identification, and detect contraband before allowing entry, Trustfile inspects digital files for malicious content before allowing user access. This preventive model contrasts with reactive approaches that respond to infections after they occur.

The simple web interface provides the primary user interaction point, designed for accessibility across technical skill levels. Web interfaces offer familiar interaction patterns—forms, buttons, progress indicators, result displays—that users encounter daily in other applications. This familiarity reduces learning curves and encourages adoption. The interface design prioritizes clarity over complexity, presenting essential functions prominently while making advanced options available but not intrusive.

File upload functionality enables users to submit files from their local computers for remote analysis. This involves browser-based file selection, secure transmission to the server, temporary storage during analysis, and result presentation. The upload process includes validation to ensure submitted files are scannable, within size limits, and in supported formats. The design for non-technical users reflects the recognition that the users most in need of file security assistance are often those with limited technical knowledge. Interface elements use plain language rather than technical jargon. Workflows require minimal steps. Results are presented with clear explanations rather than raw technical data. Help and guidance are readily available. This inclusive design ensures that security protection extends to all users, not just those with technical expertise.

The system also uses an external API to check if the file is already known as malicious. This helps improve the detection because it compares the file with existing malware databases. The external API integration represents a critical architectural component that enhances detection capabilities beyond what local analysis alone can achieve. External APIs connect Trustfile to global threat intelligence networks that maintain comprehensive databases of known malware, updated continuously through automated collection, researcher submissions, and sensor network observations.

The hash-based query mechanism typically employed for API integration preserves privacy while maximizing threat coverage. Rather than uploading complete file contents to external services, Trustfile computes a cryptographic hash of the file and transmits only this hash for database lookup. The external service returns information about whether the hash matches known malware, associated threat intelligence, and reputation scores. This approach protects sensitive file content from external exposure while leveraging comprehensive threat databases. Detection improvement through database comparison addresses the limitation of local analysis in identifying known threats. Local analysis excels at detecting novel threats through heuristic and anomaly detection but may miss known threats that match specific signatures in external

databases. The combination of local and external detection provides defense-in-depth, where each approach covers gaps in the other.

After scanning, the system shows the result by classifying the file into threat levels like LOW, MEDIUM, HIGH, or CRITICAL. The result is displayed clearly so the user can understand if the file is safe or not.

The result presentation architecture transforms raw detection data into user-comprehensible information. Classification into four threat levels provides intuitive risk communication that users can immediately understand and act upon. Each level is associated with specific colors, icons, and messages that reinforce the risk assessment.

Clear display involves organizing information hierarchically, presenting the most critical information—threat level and recommended action—prominently, with supporting details available through progressive disclosure. Visual design employs established conventions: green for safe, yellow for caution, orange for warning, and red for danger. Explanatory text describes what was found, why it matters, and what the user should do.

User understanding is facilitated by avoiding technical jargon, providing contextual explanations, and offering actionable guidance. Rather than simply stating "Threat detected," the system explains what indicators were found, what they mean in practical terms, and what steps the user should take. This educational approach builds user knowledge while providing immediate protection.

Overall, the System shows focus on making file scanning simple while still providing useful security features. It helps become more careful when opening or downloading files.

The overall architectural philosophy balances simplicity with security effectiveness. Simplicity ensures accessibility, usability, and adoption—security tools are only effective if people actually use them. Useful security features ensure comprehensive protection against diverse threat types. This balance is achieved through careful design that hides complexity behind simple interfaces while maintaining robust detection capabilities.

The behavioral impact of the system extends beyond immediate threat detection to long-term security awareness improvement. By consistently showing users what makes files dangerous, explaining threat indicators, and demonstrating safe practices, the system gradually builds security consciousness that influences file handling behavior even outside the system context. Users become more cautious about file sources, more attentive to suspicious characteristics, and more likely to seek verification before opening unexpected files.

Tools and Technologies Used

- **Frontend: HTML, CSS, JavaScript**

- The frontend technology stack encompasses the client-side technologies that create the user interface and handle browser-based interactions. HTML (HyperText Markup Language) provides the structural foundation of web pages, defining elements such as headings, paragraphs, forms, buttons, and containers that organize content and functionality. CSS (Cascading Style Sheets) controls visual presentation, defining colors, fonts, layouts, spacing, animations, and responsive behaviors that adapt to different screen sizes and devices. JavaScript provides dynamic functionality, enabling interactive elements, asynchronous server communication, form validation, dynamic content updates, and complex user interface behaviors without page reloads.

- **Backend: Python Flask**

- The backend technology selection significantly influences development productivity, runtime performance, and system capabilities. Python is a high-level, interpreted programming language renowned for readability, extensive libraries, and strong community support. Its clear syntax reduces development time and maintenance costs, while comprehensive standard libraries and third-party packages provide solutions for common programming tasks.
- Flask is a lightweight web framework for Python that provides essential web development capabilities without imposing rigid structure or unnecessary complexity. Flask's microframework philosophy means it includes only core functionality—routing, request handling, template rendering, session management—with extensions available for database integration, authentication, API development, and other needs. This minimalism provides flexibility to structure applications according to specific requirements rather than framework conventions. Flask's simplicity makes it ideal for academic projects and prototypes where development speed and clarity outweigh enterprise-scale requirements. Alternative Python frameworks like Django provide more built-in features but impose greater structure and complexity. For Trustfile's scope, Flask offers an appropriate balance of capability and simplicity.

- **Database: MySQL**

- The database technology stores persistent data including user accounts, scan histories, threat intelligence, system configurations, and audit logs. MySQL is an open-source relational database management system (RDBMS) that uses Structured Query Language (SQL) for data definition, manipulation, and querying. MySQL's popularity stems from its performance, reliability, ease of use, and extensive documentation. It supports complex queries, transactions,

indexing, and replication that enable scalable, reliable data management. For Trustfile, MySQL stores user information, file scan records, detection results, and system configuration data.

- **Development Tools: VS Code, Command Prompt, Pip**
 - Visual Studio Code (VS Code) is a free, open-source code editor developed by Microsoft that supports multiple programming languages through extensions. Features include syntax highlighting, code completion, debugging support, version control integration, and customizable themes. VS Code's popularity among developers reflects its balance of power and simplicity, extensive extension ecosystem, and cross-platform availability. The Command Prompt (Windows) or Terminal (macOS/Linux) provides command-line interface for executing development commands, running scripts, managing files, and interacting with development tools. Command-line proficiency enables efficient workflow automation and system administration. Pip is Python's package installer, enabling easy installation and management of third-party libraries and dependencies. Pip installs packages from the Python Package Index (PyPI), manages package versions, resolves dependencies, and supports virtual environments that isolate project dependencies. These tools collectively provide a complete development environment for building, testing, and deploying Python web applications.
-

Data Gathering Techniques

These are the methods the researchers used:

- **Observation** - We have observed that some companies receive tons of files every day, making them more vulnerable to malware that can harm their computers. Due to the overwhelming amount of files, they can't easily identify and separate which files may cause harm to their computers. That's where Trustfile comes in, it can scan files and let you know why they are harmful and should never be opened.

The observation of high-volume file reception reveals a critical operational challenge that amplifies malware risk. Organizations in sectors such as government, healthcare, education, and business services routinely receive hundreds or thousands of files daily through email, web portals, file sharing services, and removable media. Each file represents a potential threat vector, yet the volume makes individual scrutiny impractical. Security teams cannot manually examine every file, creating reliance on automated tools that may miss sophisticated threats.

The vulnerability created by volume extends beyond detection difficulty to include alert fatigue, where security personnel overwhelmed by numerous warnings begin to ignore or dismiss alerts, potentially missing genuine threats among false positives. Automated tools that lack contextual understanding may flag legitimate files as suspicious, consuming investigation resources and creating friction between security and operations teams.

Trustfile's value proposition addresses this observed need by providing automated scanning that processes files quickly, classifies threats accurately, and explains findings clearly. The "why they are harmful" component addresses the educational gap, ensuring that users understand threats rather than merely accepting automated verdicts. The "should never be opened" guidance provides clear, actionable direction that reduces ambiguity and supports consistent security practices.

- **Analysis of Existing Systems** - We analyzed existing platforms and systems that scan files online or through applications to observe their features and limitations. This helped us develop a better understanding that file scanning should not be the only feature, but it should also explain the problems associated with the files for the users' better understanding of harmful files.

The analysis of online scanning platforms examined services such as VirusTotal, MetaDefender, Jotti, and similar tools that provide web-based file analysis. These platforms offer comprehensive multi-engine scanning, extensive threat databases, and detailed technical reports. However, they often present results in formats accessible primarily to security professionals, with technical terminology, raw detection outputs, and minimal explanatory content that non-technical users find confusing or overwhelming.

Application-based scanning tools including antivirus software, endpoint protection platforms, and file analysis utilities were also evaluated. These tools provide deeper system integration and broader protection scope but frequently suffer from complex interfaces, intrusive notifications, performance impacts, and opaque detection processes that alienate users.

The key insight from this analysis is that detection alone is insufficient for effective security. Users who do not understand why files are dangerous cannot make informed decisions about similar files in the future, cannot verify detection accuracy, and may distrust or circumvent security tools that seem arbitrary or opaque. The educational component—explaining problems associated with files—transforms security from a black-box service into a learning opportunity that builds long-term user resilience.

This understanding directly shaped Trustfile's design priorities, ensuring that explanatory content, educational context, and clear communication are integral features rather than afterthoughts. Every detection result includes explanation of what was found, why it matters, and how users can protect themselves, addressing the educational gap identified through analysis of existing systems.

Testing Procedures

In order to make TRUSTFILE work properly as intended, here are the types of test we conducted:

- **Unit Testing** - This phase focused on the functionality of individual components such as the file hashing module, entropy analysis, script scanner, file type verifier, and risk identifier, using both valid and malicious test samples to confirm accurate threat detection.

The file hashing module testing validates that hash computation produces correct, consistent results for known inputs. Tests include verifying that identical files produce identical hashes, that different files produce different hashes (collision resistance), that hash algorithms execute correctly across file sizes, and that hash comparison against threat databases functions properly. Test samples include files with known hashes to verify correctness.

Entropy analysis module testing confirms that entropy calculation accurately measures randomness, that baseline profiles correctly represent normal file types, and that anomaly detection flags appropriately deviant files. Tests include files with known entropy characteristics—highly compressed files, encrypted files, plain text files, structured binary files—to validate measurement accuracy and threshold calibration. Script scanner module testing verifies detection of embedded executable code across file formats. Tests include PDFs with JavaScript, Office documents with macros, archives containing scripts, and files with encoded or obfuscated scripts. Both positive cases (files containing scripts) and negative cases (files without scripts) validate detection accuracy and false positive rates.

File type verifier module testing confirms that magic number detection, structure validation, and extension comparison correctly identify actual file types. Tests include correctly labeled files, files with spoofed extensions, polyglot files, truncated files, and files with corrupted headers to validate type detection across edge cases.

Risk identifier module testing validates that classification logic correctly combines individual indicator results into overall threat assessments. Tests include files with single indicators, multiple indicators, conflicting indicators, and no indicators to verify that classification boundaries and weighting schemes produce appropriate results.

The use of both valid and malicious test samples ensures comprehensive validation across the full spectrum of files the system will encounter. Valid samples test that the system does not falsely flag legitimate files, while malicious samples test that genuine threats are detected. This dual testing approach quantifies both false positive and false negative rates, essential metrics for evaluating detection system quality.

- **System Testing** - In this phase we focused on how the system scanned files, if it scanned properly and identified what level of threat is being scanned and, it assesses overall performance of the system as a whole to ensure that the system works properly as intended.

Scanning workflow testing validates the complete process from file upload through result presentation. This includes verifying that uploaded files are correctly received, stored, and processed; that scanning completes successfully across different file types and sizes; that results are accurately computed and properly formatted; and that result presentation is timely and comprehensible. Workflow testing identifies integration issues that unit testing cannot detect, such as data format incompatibilities between components, timing issues in asynchronous operations, and resource conflicts in shared environments.

Threat identification accuracy testing evaluates whether the system correctly identifies and classifies threats across the full range of test files. This involves scanning files with known characteristics and verifying that assigned threat levels match expected classifications. Accuracy testing quantifies system performance through metrics including detection rate, false positive rate, false negative rate, and classification consistency.

Overall performance assessment examines system behavior under realistic operating conditions. This includes measuring response times for file scanning, evaluating throughput under concurrent user loads, assessing resource consumption (CPU, memory, disk, network), and verifying stability during extended operation. Performance testing ensures that the system meets usability expectations and can handle anticipated workloads without degradation.

The "works properly as intended" criterion encompasses functional correctness, performance adequacy, reliability under stress, security against attacks, and usability for target users. System testing validates each of these dimensions, providing comprehensive evidence that the system is ready for deployment and use.

- **User Acceptance Testing** - This phase includes real users or staff to upload files and validate systems user interface, report clarity, and overall experience, gathering feedback if the system performed effectively and educates the users about file security risks.

Real user involvement ensures that evaluation reflects actual user perspectives rather than developer assumptions. Users bring diverse backgrounds, technical skill levels, work contexts, and expectations that reveal issues invisible to developers. Their feedback identifies usability problems, confusing interfaces, missing features, and workflow mismatches that technical testing cannot detect.

File upload validation by real users tests whether the upload process is intuitive, efficient, and reliable across different browsers, devices, and network conditions. Users attempt uploads using their own files, revealing issues with file type support, size limits, error handling, and progress feedback that controlled testing might miss.

User interface validation assesses whether the interface design effectively supports user tasks. This includes evaluating navigation clarity, information findability, action discoverability, visual hierarchy, and aesthetic appeal. Users attempt typical tasks—scanning files, interpreting results, accessing help—while observers note difficulties, confusions, and delays.

Report clarity evaluation examines whether scan results are comprehensible and actionable for non-technical users. Users interpret results without assistance, revealing whether threat levels are clear, explanations are understandable, and recommended actions are feasible. Feedback on report clarity directly informs improvements to result presentation and educational content.

Overall experience assessment captures holistic impressions that transcend individual features. Users evaluate whether the system feels trustworthy, professional, helpful, and worth using. These subjective assessments influence adoption and sustained use, determining whether the system achieves its security objectives in practice.

Feedback gathering on educational effectiveness examines whether the system actually improves user security awareness as intended. Users report whether explanations increased their understanding, whether they feel better equipped to recognize threats, and whether their file handling behavior has changed. This feedback validates the educational component of Trustfile's design and identifies opportunities for improvement.

Evaluation Criteria

Trustfile will be evaluated based on the following criteria:

- **Functionality** - This criteria will assess if the system performed all of its main features properly, including file scanning, file analysis, threat classification, and explained properly why the file was harmful. All parts of the system must work as expected without errors.

File scanning functionality is evaluated by verifying that the system successfully receives, processes, and analyzes uploaded files across supported types and sizes.

Tests include normal files, large files, small files, files with special characters in names, and files in various formats. Success criteria include successful upload completion, scanning initiation, analysis execution, and result generation.

File analysis functionality is evaluated by confirming that each analysis method—hashing, entropy, pattern detection, type verification—produces correct results for test files with known characteristics. Evaluation compares system outputs against expected outputs for validated test datasets, quantifying accuracy and identifying discrepancies.

Threat classification functionality is evaluated by verifying that classification logic correctly assigns threat levels based on analysis results. Tests include files with known characteristics that should trigger specific classifications, files with ambiguous characteristics that test boundary conditions, and files with no suspicious characteristics that should receive safe classification.

Explanation functionality is evaluated by assessing whether generated explanations accurately describe detected threats, correctly explain technical concepts in accessible language, and provide actionable guidance. Evaluation involves expert review of explanation accuracy and user comprehension testing to verify accessibility.

The "without errors" requirement establishes that functional correctness includes robust handling of unexpected inputs, edge cases, and error conditions. The system should not crash, hang, or produce incorrect results when encountering malformed files, network interruptions, service unavailability, or other exceptional conditions.

- **Usability** - This criteria will measure if the system is easy to use, even for non-technical users. Users should be able to upload and scan files easily, and the results should be clear and understandable without needing instructions.

Ease of use for non-technical users is evaluated through task-based testing where representative users attempt typical workflows without assistance. Observers measure task completion rates, time to completion, error rates, and help requests. Users provide subjective ratings of difficulty and confidence. Low completion rates, long completion times, frequent errors, or low confidence indicate usability deficiencies.

Upload and scan ease is evaluated by examining the number of steps required, the clarity of instructions, the intuitiveness of interface elements, and the feedback provided during the process. Ideal workflows minimize required actions, use familiar interaction patterns, provide clear progress indication, and confirm successful completion.

Result clarity and understandability is evaluated by testing whether users correctly interpret scan results without reference materials. Users explain what they understand from results, identify the threat level, describe what was found, and state what action they should take. Misinterpretations, confusion, or inability to extract key information indicate presentation deficiencies.

The "without needing instructions" criterion establishes a high usability standard—the system should be sufficiently intuitive that first-time users can accomplish tasks successfully without reading manuals, watching tutorials, or receiving training. This standard reflects the reality that users rarely consult documentation and expect immediate usability.

- **Accuracy of Detection** - This criteria will evaluate the systems accuracy and reliability of file scanning, it can identify suspicious and potentially malicious files while minimizing false results.

Detection accuracy is measured through precision (the proportion of positive predictions that are correct), recall (the proportion of actual positives correctly identified), and F1-score (the harmonic mean of precision and recall). High precision indicates few false positives, while high recall indicates few false negatives. The F1-score balances these concerns.

Identification of suspicious and potentially malicious files is evaluated using test datasets with known labels. The system scans labeled files and its predictions are compared against ground truth. Accuracy is calculated across different file types, threat categories, and obfuscation techniques to ensure comprehensive evaluation.

Minimizing false results encompasses both false positives (benign files incorrectly flagged as threats) and false negatives (malicious files incorrectly classified as safe). False positives cause unnecessary concern, operational disruption, and potential circumvention of security measures. False negatives allow threats to evade detection, exposing systems to compromise. The optimal balance depends on organizational risk

tolerance, with high-security environments accepting more false positives to minimize false negatives.

Reliability evaluation assesses consistency of detection across repeated scans, different system configurations, and varying operating conditions. A reliable system produces consistent results for identical inputs, maintains accuracy under load, and degrades gracefully when components fail.

- **Security** - This criteria will ensure that everything is secured and can safely handle files during scanning. Files are only read and analyzed, not executed or opened, to prevent possible infection if the file is flagged as a threat. The system should ensure that scanning does not harm the user's device.

Secure handling of files during scanning involves ensuring that scanning processes cannot be exploited to compromise the scanning system or exfiltrate data. This includes sandboxing analysis processes to limit their access to system resources, validating file contents before processing to prevent parser exploits, restricting network access during analysis to prevent data exfiltration, and sanitizing outputs to prevent injection attacks.

Read-only analysis without execution is a critical security principle for static analysis systems. By examining file contents without executing them, the system avoids triggering malicious payloads that could infect the scanning infrastructure. This principle is implemented through careful parser design that extracts and analyzes file structures without invoking executable content, sandboxed environments that isolate potential execution, and monitoring that detects and prevents unauthorized execution attempts.

Protection of user devices extends beyond the scanning process to encompass the entire system interaction. The web interface should prevent cross-site scripting (XSS) attacks that could compromise user browsers. File uploads should be protected against path traversal, buffer overflow, and denial of service attacks. Data transmission should be encrypted to prevent interception. User credentials should be securely stored and transmitted.

The system should not harm user devices through resource exhaustion, unauthorized modifications, or introduction of vulnerabilities. This includes managing resource consumption to prevent denial of service, avoiding persistent changes to user systems, and ensuring that the system itself does not become a vector for malware distribution.

- **Efficiency** - This criteria will assesses the performance of file scanning within a reasonable time. It should process and scan files quickly and display results without delays to ensure fast and smooth user experience.

Reasonable scanning time is context-dependent, varying with file size, analysis complexity, and user expectations. For typical documents, users expect results within seconds. For larger files or comprehensive analysis, slightly longer times may be acceptable if progress is indicated. Evaluation establishes benchmarks based on user expectations and competitive products, then measures actual performance against these benchmarks.

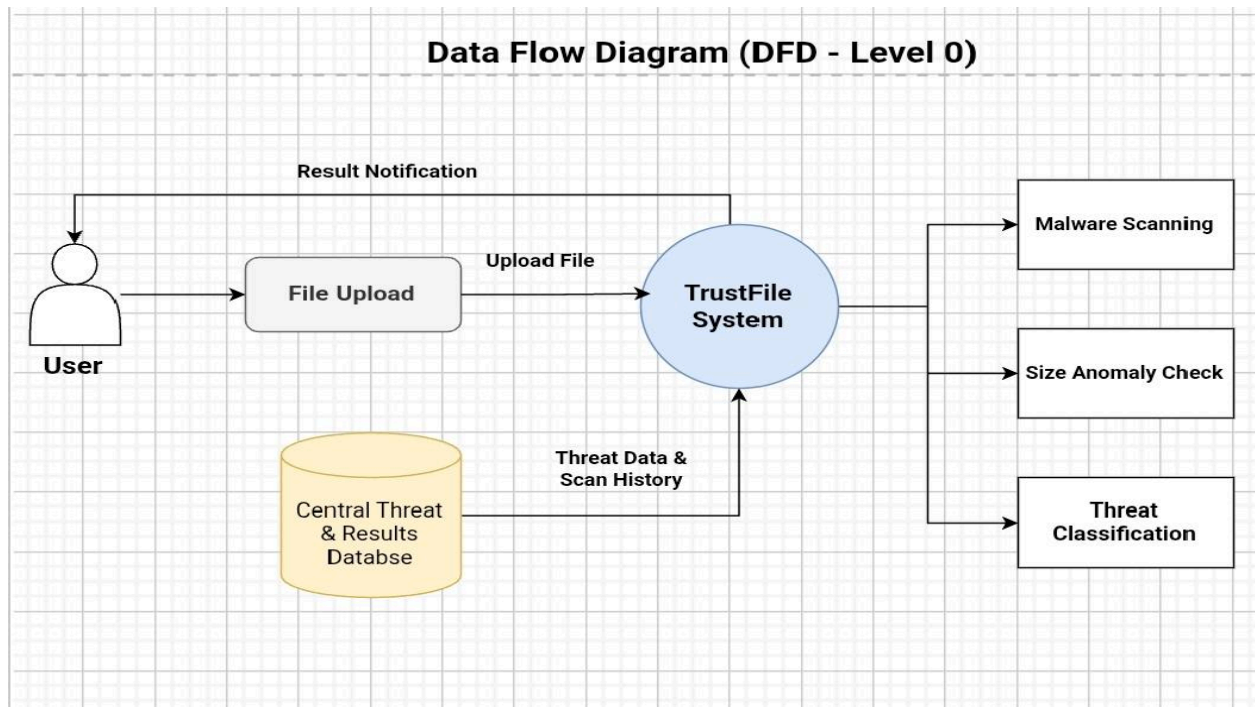
Quick processing and scanning involves optimizing algorithms, leveraging caching, parallelizing operations, and minimizing unnecessary computation. Performance profiling identifies bottlenecks, while optimization techniques address identified slowdowns. The goal is sub-second response for typical files with graceful degradation for larger or more complex inputs.

Display without delays requires efficient result formatting, transmission, and rendering. Results should be available immediately upon analysis completion, with presentation optimized for rapid comprehension. Delays between analysis completion and result display frustrate users and suggest system inefficiency.

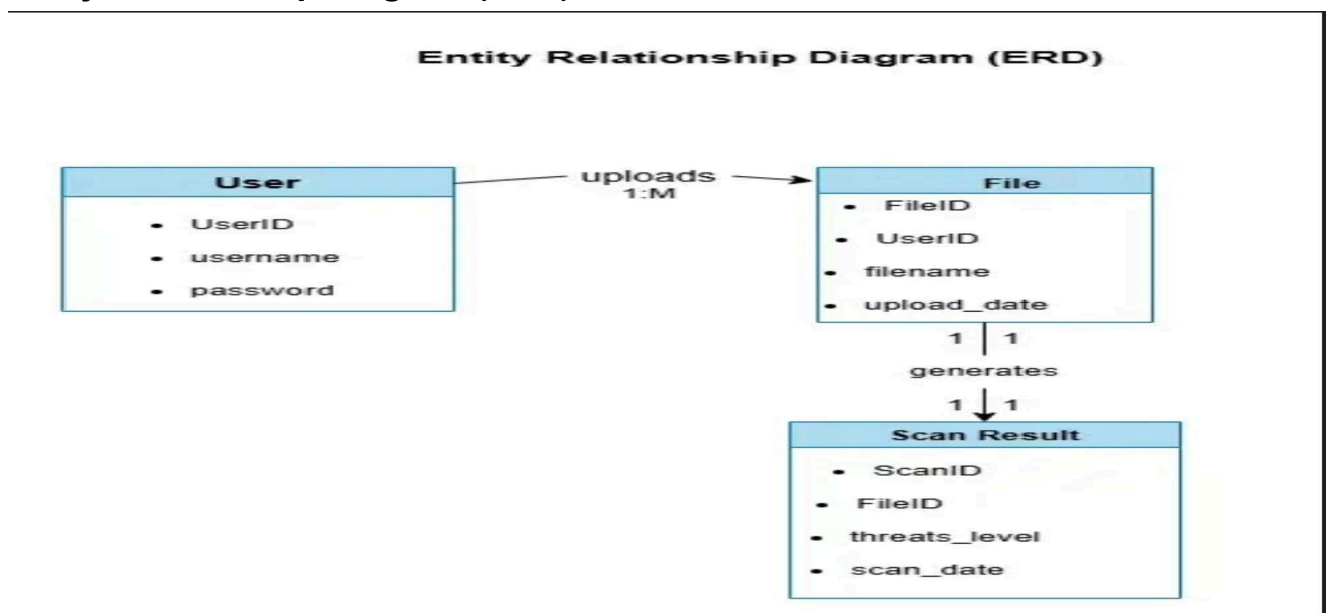
Fast and smooth user experience encompasses not only scanning speed but overall system responsiveness. Interface interactions should feel immediate, transitions should be fluid, and feedback should be timely. Performance optimization addresses server response times, database query efficiency, network latency, and client-side rendering to ensure comprehensive responsiveness.

SUPPORTING DIAGRAMS

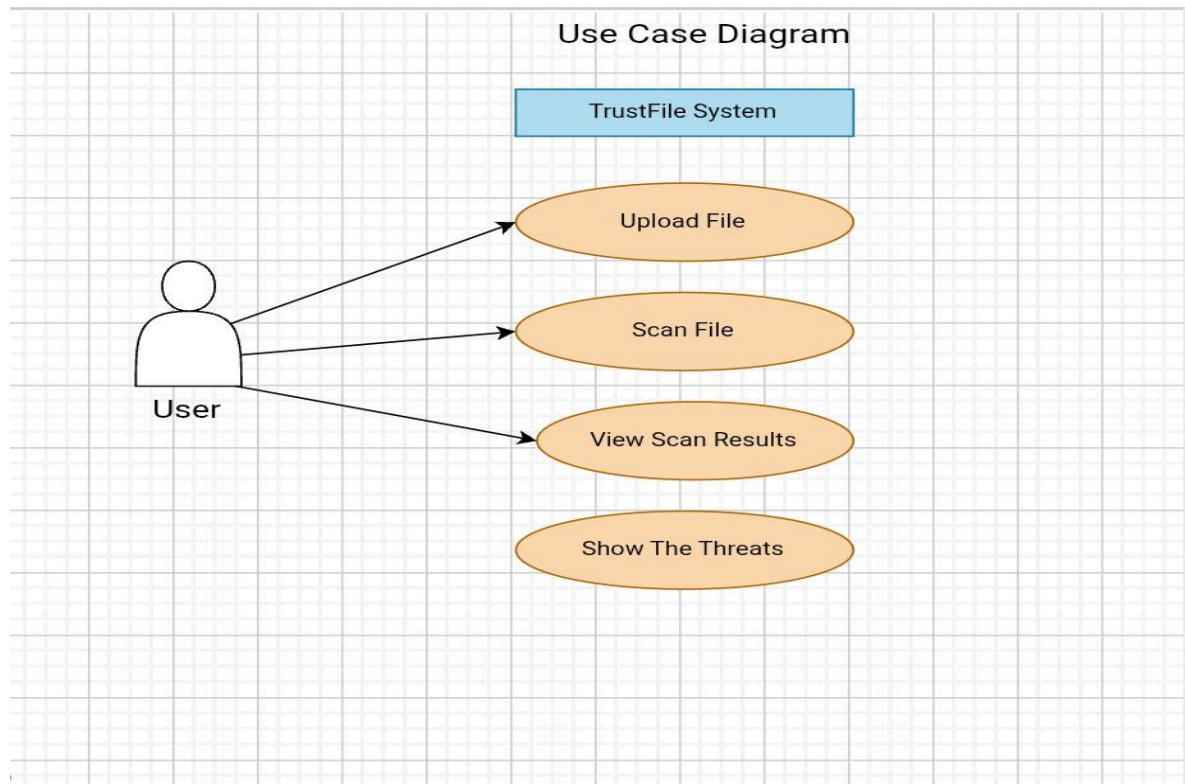
Data Flow Diagram (DFD – Level 0)



Entity Relationship Diagram (ERD)



Use Case Diagram



References

- Ö. A. Aslan and R. Samet, "A Comprehensive Review on Malware Detection Approaches," in IEEE Access, vol. 8, pp. 6249-6271, 2020, doi: 10.1109/ACCESS.2019.2963724.
[A Comprehensive Review on Malware Detection Approaches | IEEE Journals & Magazine | IEEE Xplore](#)
- Aaron Lensmer Abdon. (2024). RATTATA: Ransomware Analysis via Automated Impact Assessment of Philippine Organizations. *International Journal of Intelligent Systems and Applications in Engineering*, 12(4), 3430 –. Retrieved from <https://ijisae.org/index.php/IJISAE/article/view/6858>
- Caluza, L. J. B. (2024). Harnessing AI and analytics to enhance cybersecurity and privacy for collective intelligence systems. *Journal of Big Data and Cognitive Computing*. (Republished via PMC11419604). <https://pmc.ncbi.nlm.nih.gov/articles/PMC11419604/>